

Golden: Lightweight Non-Interactive Distributed Key Generation

Benedikt Bünz¹, Kevin Choi^{2*}, and Chelsea Komlo³

¹ New York University

² Seoul National University

³ University of Waterloo & NEAR One

bb@nyu.edu, kc2296@nyu.edu, ckomlo@uwaterloo.ca

Abstract. In this work, we present **Golden**, a non-interactive Distributed Key Generation (DKG) protocol. The core innovation of **Golden** is how it achieves public verifiability in a lightweight manner, allowing all participants to non-interactively verify that all other participants followed the protocol correctly. For this reason, **Golden** can be performed with only one round of (broadcast) communication. Non-interactive DKGs are important for distributed applications; as parties may go offline at any moment, reducing rounds of communication is a desirable feature.

Golden outputs Shamir secret shares of a field element $sk \in \mathbb{Z}_p$ to all participants, and a public key $PK = g^{sk}$ that is a discrete-logarithm commitment to sk . Further, the security of **Golden** requires only the hardness of discrete-logarithm assumptions, and so can be used over any elliptic curve where these assumptions hold.

Golden is more efficient than prior related schemes in both bandwidth and computation. For 50 participants, **Golden** requires only 223 kb bandwidth and 13.5 seconds of total runtime for each participant, in comparison to ElGamal-based non-interactive DKG, which requires 27.8 MB bandwidth and 40.5 seconds runtime per participant.

As a building block, we define a new exponent Verifiable Random Function (eVRF). Our eVRF uses a non-interactive key exchange (NIKE) as a building block to derive a Diffie-Hellman shared secret key, and proves correctness of this key with respect to the corresponding Diffie-Hellman public keys. Within **Golden**, participants use this eVRF in a pairwise manner to generate a one-time pad to encrypt Shamir secret shares to their respective recipients while ensuring public verifiability. As such, **Golden** avoids the use of public-key encryption schemes such as ElGamal, Paillier, or class groups, departing from prior schemes in the literature. Finally, our eVRF may be of independent interest to settings where publicly-verifiable encryption is desirable.

* Most work done while at NYU

Table of Contents

1	Introduction	3
1.1	Our Contribution	4
1.2	Case Studies	8
2	Additional Related Work	10
3	Preliminaries	12
3.1	Distributed Key Generation (DKG) with Additive Bias	12
3.2	Exponent Verifiable Random Functions	14
	Game-Based Notions of Security	15
	Simulation-Based Notion of Security	16
3.3	Polynomial Interpolation and Shamir Secret Sharing	16
3.4	The Bulletproofs Argument System	17
4	A New Exponent Verifiable Random Function	17
4.1	Argument System for the Relation $\mathcal{R}_{\text{eVRF}^\dagger}$	20
4.2	First Gadget: Bit-Decomposition Check	20
4.3	Second Gadget: Exponentiation Check	20
4.4	No Need to Arithmetize the Last Exponentiation	22
4.5	Combining Everything for $\mathcal{R}_{\text{eVRF}^\dagger}$	22
4.6	Concrete Efficiency, Batch Proving, and Batch Verification	22
5	Golden: Lightweight Non-Interactive Distributed Key Generation	24
5.1	Requirement of a Discrete-Log Based PKI	24
5.2	Protocol Description	24
5.3	Optimized Golden via Batching	27
6	Security	27
A	Simulation-Based Security for eVRFs	35
B	Non-Interactive Key Exchange (NIKE)	35
C	Leftover Hash Lemma	37
D	Proof of Theorem 2	38
E	A Different Approach in Proof Construction	41
F	Public Key Infrastructure	41
F.1	Idealized Functionality $\mathcal{F}_{\text{pki}}$	41
F.2	A Concrete Protocol Realizing $\mathcal{F}_{\text{pki}}$	42
G	Simulation-Based Security for $\mathcal{R}_{\text{pki}}$	43
H	Proof for Theorem 3	43

1 Introduction

Distributed key generation (DKG) is a fundamental building block for multi-party applications, as DKGs allow n parties to cooperate to generate a secret key, which all parties have contributed to but no single participant knows, yet any t parties can cooperate to recover for some threshold parameter $t \leq n$. DKGs underpin threshold cryptosystems such as threshold encryption [32, 33], threshold PRFs [51], and threshold signature schemes [9, 56, 59]. Furthermore, many secure distributed applications build upon DKGs, including distributed randomness beacons [20, 25, 55, 58, 72], distributed secret storage applications [73], and consensus protocols [34, 60]. Threshold schemes are currently undergoing increased use in practice, and are the subject of ongoing standardization efforts [13]. As such, real-world considerations, such as reducing round complexity, computational and bandwidth efficiency, and protocol simplicity are of increased importance.

Limiting round complexity is of particular importance, due to the complexity of managing interaction among distributed parties. In a distributed environment, parties may fail to send or receive messages due to network issues. For distributed applications that require either a large number of participants (as in consensus protocols) or frequent share rotation, limiting rounds of participant interaction improves the probability of protocol success. For this reason, while interactive (multi-round) DKGs exist in the literature [46, 47, 49, 56], non-interactive (one-round) DKGs are of particular interest. They allow participants to go offline after posting a single message and do not require stable connections or IP addresses (to compute their key share they still need to receive messages but this can happen at an arbitrary later point). However, in spite of their desirability, non-interactive DKGs incur significant complexity over interactive (multi-round) DKGs, for reasons we explore next.

Public Verifiability Versus Round Complexity. One difference between non-interactive and interactive DKGs is the requirement for *public verifiability*. In particular, in the non-interactive DKG setting, participants must be able to verify the correctness of the protocol for all participants without incurring additional rounds of communication. More specifically, DKGs must prevent against “splitting” attacks, where an adversary might send valid contributions to some parties but invalid contributions (either malformed or vacuous) to other parties, resulting in a denial-of-service attack. Interactive DKGs prevent against such attacks by having participants broadcast complaints if any other player misbehaves. In the non-interactive setting, it must be possible to verify correctness without incurring additional rounds of communication.

Existing non-interactive DKGs in the literature achieve public verifiability by following a similar approach using public-key encryption based on ElGamal [48], Paillier [42], or class groups [19, 53], along with zero-knowledge proofs to prove correctness. However, the use of public-key encryption introduces downsides. In the case of ElGamal encryption, public-key decryption must be done over a uniformly random field element, incurring significant performance overhead. In the

case of Paillier or class-group-based encryption, additional security assumptions are required (if the DKG outputs discrete-logarithm key material).

Alternative approaches to generating keys in a distributed manner exist in the literature, but with the tradeoff of increased round complexity. One approach is to perform a one-time setup using a multi-round protocol, after which participants can generate subsequent distributed keys non-interactively [54]. A second approach is to use private channels to distribute secret shares, while sending public commitments and proofs of correctness over a public broadcast channel [12]. However, the use of private channels prevents public verifiability, and thus necessitates a second consensus round for participants to agree on correctness, as an adversary can simply send valid shares to some participants and invalid shares to other participants.

We give a concrete comparison of our techniques with two case studies in the literature most related to our work, in Section 1.2.

1.1 Our Contribution

In this work, we present **Golden**⁴, a novel non-interactive distributed key generation scheme that does not require public-key encryption, and so offers efficiency improvements over prior non-interactive DKGs in the literature. We show a comparison of **Golden** and related schemes in the literature in Table 2. We give an asymptotic comparison of the performance of **Golden** in Table 2, and a concrete comparison to Groth’s DKG [48] in Table 1.

Golden uses the standard approach of each participant i generating Shamir secret shares of a random field element $\omega_i \in \mathbb{Z}_p$, and then distributing shares to all other participants. The output secret key of the DKG is simply $\text{sk} = \sum_{j=1}^n \omega_j$, and the public key is g^{sk} . Furthermore, at the end of the protocol, each participant holds a secret key share sk_i that is a Shamir secret share of sk , which is the sum of their received shares $\text{sk}_i = \sum_{j=1}^n f_j(i)$, such that $f_j(0) = \omega_j$.

The core innovation of **Golden** is a new technique to achieve public verifiability; i.e., how each participant j transmits $f_j(i)$ to its intended recipient i , such that only participant j learns its value, but all other participants can verify its correctness with respect to the polynomial commitment to f_j .

A New Exponent Verifiable Random Function (eVRF). As a building block towards **Golden**, we introduce a new exponent verifiable random function (eVRF) [12]. First formalized by Boneh et al. [12] but used in prior work by Nick et al. [62], an eVRF is simply a verifiable random function (VRF) [35] that outputs a tuple (α, A, π) , where α is a random value, $A = g^\alpha$ is a commitment to α , and π is a zero-knowledge proof that α was correctly generated.

We introduce a new eVRF, which uses a non-interactive key exchange (NIKE) as a building block to derive a Diffie-Hellman shared secret key, and proves correctness of this key with respect to the corresponding Diffie-Hellman public keys. As such, our eVRF allows any two parties with the corresponding public

⁴ Silence is golden, and the goal of this work is to minimize communication overhead.

Participants	Golden (optimized)		Groth DKG [48]	
	Communication	Runtime	Communication	Runtime
2	1.7 kb	0.4 s	59.4 kb	3.9 s
10	22 kb	2.4 s	1.3 MB	5.6 s
50	223 kb	13.5 s	27.8 MB	40.5 s
100	699 kb	35.8 s	108.6 MB	130.8 s

Table 1: Benchmarks for the optimized variant of **Golden** and Groth’s DKG [48]. Estimates for Groth’s DKG are done using benchmarks and implementation from [53]; estimates for **Golden** use zkalc [37]. Because the Groth implementation [53] uses BLS12-381, we likewise estimate **Golden** for this curve. We consider running the DKG in an n -of- n configuration; kb represents kilobytes, and MB represents megabytes. Runtime represents the running time for each participant of performing both Round 0 and Round 1.

keys to derive the *same* random output, while ensuring public verifiability of the correctness of this output with a zero-knowledge proof.

At a high level, our eVRF is based on the Decisional Diffie-Hellman (DDH) assumption. Evaluating the eVRF requires as input a message m , a secret key sk_1 , and a public key PK_2 . Generating the random output α involves the following (simplified) steps. First, the function finds the Diffie-Hellman shared secret $S \leftarrow \text{PK}_2^{\text{sk}_1}$ and derives the x -coordinate $k \leftarrow S.X$, where “. X ” indicates the operation to output the x -coordinate. Then, the function derives $T \leftarrow H(m)^k$, where H is a random oracle. Finally, the function again derives the x -coordinate $\alpha \leftarrow T.X$. The commitment is simply $A \leftarrow g^\alpha$, where g is the generator for a prime-order group, and π is a zero-knowledge proof generated with the Bulletproofs argument system [15]. Verifying the eVRF requires as input the message m , public keys $(\text{PK}_1, \text{PK}_2)$, the commitment A , and the proof π .

The efficiency of our eVRF is bounded by the efficiency to generate and verify π . The Bulletproofs argument system outputs a proof for 3598 constraints (for security parameter $\lambda = 256$), whose computation is dominated by a multi-scalar multiplication (MSM) of size 10794 and two MSMs of size 8192. The proof consists of 29 group elements, and the verifier’s computation is dominated by an MSM of size 14419. We estimate the performance using zkalc [37] to be 0.3 and 0.1 seconds for proving and verification, respectively. While these are for one eVRF with one proof only, we provide more details and multiple batching techniques in Section 4.6.

Distributed Key Generation via Golden. Building on this new eVRF, **Golden** is defined as follows. All parties begin by performing a one-time setup operation, by first generating an identity public key. Then, each party i will register their identity public key PK_i with a central authority, providing a zero-knowledge proof of the corresponding private key.

Scheme	Rounds	Bandwidth	Computation	Assumption
Fouque & Stern [42]*	1	$\mathcal{O}(n^2t)$	$\mathcal{O}(nt)$	Paillier
ElGamal-DKG/Groth [48]	1	$\mathcal{O}(n^2tcl)$	$\mathcal{O}(ntcl)$	DL/DDH
Cascudo & David [19] Kate et al. [53]	1	$\mathcal{O}(n^2t)$	$\mathcal{O}(nt)$	Class Groups
Katz [54]	3 (2 preprocessing)	$\mathcal{O}(n^2t)$	$\mathcal{O}(\binom{n}{t})$	DL
BHLS25-DKG [12]	$2^\dagger$	$\mathcal{O}(n^2t)$	$\mathcal{O}(nt)$	DL/DDH
Golden (this work)	1	$\mathcal{O}(n^2t)$	$\mathcal{O}(nt)$	DL/DDH

Table 2: Comparison of performance overhead of Golden to prior DKGs that output discrete-log based keys, where the resulting secret key is Shamir secret shared among all participants. Here, n is the total number of parties, t is the threshold, and c, ℓ are constants such that $c = \lambda/\ell$, where λ is the security parameter. † indicates that an additional round of communication is required for participants to agree on the output status of the protocol (success or abort). $*$ indicates that the scheme does not output discrete-logarithm key material, but we include for comparison of the scheme’s relative round complexity.

When performing the DKG, all participants perform the role of a verifiable secret sharing (VSS) dealer. Specifically, each participant i samples a secret ω_i uniformly at random, and then generates Shamir secret shares $f_i(j)$ of this secret, where f_i is a random polynomial defined by Shamir secret sharing. Additionally, each participant generates a VSS commitment to f_i . Next, each party samples a message m_i uniformly at random, and then for all other parties $j \neq i$, party i uses our new eVRF to (deterministically) generate a uniformly random output $r_{i,j}$ by evaluating the eVRF on (m_i, PK_j) with sk_i , as well as a commitment output $R_{i,j} = g^{r_{i,j}}$ and an eVRF proof. This eVRF output $r_{i,j}$ is then used as a one-time pad to derive the ciphertext $z_{i,j} \leftarrow r_{i,j} + f_i(j)$, a linear encryption of party j ’s Shamir secret share. Party i then broadcasts its VSS commitment to f_i , the message m_i , and all ciphertexts $(R_{i,j}, z_{i,j})_{j=1}^n$ to all other parties.

After receiving VSS commitments, ciphertexts, and eVRF proofs from all other parties, each party first checks that the eVRF proof is correct. Then, they check that the encryptions of all shares are correct, using the eVRF commitment output in conjunction with the VSS commitment. Because this check is a linear operation (as it simply checks a linear relation “in the exponent”), this check can be done efficiently. If all checks are correct, each party simply aggregates their received shares $\text{sk}_i = \sum_{j=1}^n f_j(i)$ to derive their output secret key share; public keys are derived using the VSS commitments.

As such, **Golden** achieves public verifiability through the combination of two correctness checks:

- (1) the correctness check that each participant's received secret share $f_i(j)$ is valid, using the ciphertext $z_{i,j} \leftarrow r_{i,j} + f_i(j)$, the eVRF commitment output $R_{i,j}$, and the VSS commitment to f_i , and
- (2) the correctness check that the ciphertext $z_{i,j}$ is valid and participant j can indeed decrypt it, using the eVRF proof to check $r_{i,j}$ was derived correctly with respect to the message m_i . Because our eVRF uses a NIKE to derive $r_{i,j}$, this check guarantees that either participant i or participant j can perform the eVRF evaluate function to generate $r_{i,j}$.

Performance of Golden and Optimizations. We give an asymptotic comparison of the performance of **Golden** in Table 2, and a concrete comparison to Groth's DKG [48] in Table 1. Similarly to other DKGs, as the number of parties n grows large, the naive performance of **Golden** by each party in **Golden** scales quadratically in n , as each party must receive $(n - 1)^2$ messages, and check the correctness of all parties' contributions, as discussed above. In the unoptimized variant of **Golden**, the dominant computational cost is for each participant to perform $(n - 1)$ eVRF evaluate operations, and then verify $(n - 1)^2$ eVRF proofs.

However, it is possible to define **Golden** in an optimized manner, by batching the most expensive computation performed by each party (generating eVRF proofs), and hence performing this costly computation only once. More specifically, each party will generate only one (as opposed to $n - 1$) proof that all their eVRFs are correctly evaluated. When instantiated with Bulletproofs, the prover time still grows quasi-linearly in n ($n / \log(n)$). However, the proof size, and thus the overall communication complexity, now drastically decreases, as each party sends only one proof whose size is essentially independent of n . A proof for 1 statement is of size 1.5 kb, using BLS12-381, and a batch proof for 99 statements is only 2.2 kb. In addition to the proof, each party also sends t group elements representing the polynomial that encodes their contribution. For large n and t , this becomes the communication bottleneck. The total receiving communication per participant for a 2-out-of-2 DKG is 1.7 kb and for a 100-out-of-100 DKG it is 699 kb.

In addition to batch proving, Bulletproofs supports batch verification [15], where the cost of checking an MSM can be shared across proofs. Using this, the total runtime for each participant in the DKG application, which includes one proof generation and $n - 1$ proof verifications, is estimated in Table 5, and we estimate the total runtime for 100 participants to be less than 36 seconds.

UC Security of Golden. We prove simulation-based security of **Golden** with respect to an idealized key generation scheme $\mathcal{F}_{\text{KeyGen}}^\Delta$. To establish the proof, we assume the use of a public-key infrastructure (PKI) modeled by an ideal functionality $\mathcal{F}_{\text{pki}}$, where participants are required to provide a proof of knowledge of their corresponding secret key when registering their identity public key. Because our proof is straight-line, by extension, we can guarantee security under composition in the Universal Composability (UC) framework [57].

While Katz [54] gives an impossibility result that shows one-round DKGs cannot produce unbiased keys that are from the same distribution as a target single-party key generation algorithm (such as discrete log, in the case of **Golden**),

we show that **Golden** can achieve a slightly weaker but still relevant notion of security. In particular, we show that **Golden** realizes the functionality $\mathcal{F}_{\text{KeyGen}}^\Delta$, which considers a rushing adversary (one which chooses to speak last in the protocol) that can bias the key by some additive value Δ , but only in a limited way; i.e., by choosing their contributions to the DKG as a function only of honest participants' broadcast messages.

Many DKGs used in practice achieve the same $\mathcal{F}_{\text{KeyGen}}^\Delta$ functionality. For example, Pedersen DKG [66] is a two-round DKG, where all participants publish their VSS commitments in the clear, without a commitment round. Hence, a rushing adversary can choose its contribution of the DKG as a function of honest parties' VSS commitments. However, intuitively, this advantage to an adversary is bounded, assuming the VSS commitments are generated within a group where the discrete logarithm problem is hard. Towards this end, it has been shown that Pedersen DKG with proofs of possession [56] is safe to use as a DKG for the FROST threshold signature scheme [5, 30, 56]. Similarly, the ElGamal-based DKG by Groth [48] (which we discuss more in Section 1.2) also allows for additive bias, as parties likewise publish their VSS commitments in the clear (as the DKG is only one round). However, Groth shows that this DKG can be safely used in the context of threshold BLS [48] (and hence, as the DKG for randomness beacons [58]). We postulate that $\mathcal{F}_{\text{KeyGen}}^\Delta$ can be used to securely build other applications, such as threshold encryption.

Because the Katz [54] impossibility rules out only full security for one-round DKGs, it may be possible that using our techniques in combination with non-binding VSS commitments (such as Pedersen commitments) for the polynomial commitment could achieve a stronger notion of security (such as security with aborts). However, doing so would require more involved zero-knowledge proofs for demonstrating share validity, and hence incur performance and complexity overhead. We leave this exploration to future work.

1.2 Case Studies

To illustrate the limitations of prior work, we highlight two case studies that are most relevant to our work. Similarly to **Golden**, in both of the below case studies, each participant plays the role of a Shamir secret sharing dealer, where sk is a Shamir-secret shared element in $\mathbb{Z}_p$. The key difference in the below case studies is how participants arrive at consensus on the correctness of the protocol. The case studies differ on how they chose to distribute $f_j(i)$ such that only player j learns its value, while ensuring all other participants can verify its correctness.

Case Study One: ElGamal-DKG and Range Proofs. One possible approach to building a non-interactive DKG is to use public-key encryption, and then generate zero-knowledge proofs that the encryption was performed correctly. This approach was taken by Groth [48], who used ElGamal encryption and range proofs in a pairing setting, such that the DKG outputs a BLS public key in a pairing group. Groth proves security of the DKG with respect to the threshold BLS signature

scheme. However, a DKG following the same approach could be used without pairings, which is why we use the term ElGamal-DKG for broader context.

Recall that to encrypt a message m to a public key PK using ElGamal, the ElGamal ciphertext is the tuple $C = (C_0, C_1)$ is such that $C_0 = g^r$, and $C_1 = g^m \cdot PK^r$, where $r \leftarrow \mathbb{Z}_p$ is a random nonce. Decryption requires first deriving $H' \leftarrow C_0^{\text{sk}}$, and then inverting $f^{-1}(M)$. As such, the efficiency of ElGamal encryption is bounded by the ability to efficiently invert $f^{-1}(M)$.

In ElGamal-DKG, each party j produces an ElGamal ciphertext that encrypts each party's Shamir share $f_j(k)$ to party k 's public key. Each participant then broadcasts all ElGamal ciphertexts, along with zero-knowledge proofs that the encryption was performed correctly. To complete the DKG protocol, each party k must then perform $n - 1$ ElGamal decryptions. However, performing ElGamal decryption naively would require each party to find the discrete logarithms of the group elements $g^{f_j(k)}$ for $j \in [n]$, a computationally prohibitive task.

To enable efficient decryption, Groth instead uses a chunking approach, to split the bits representing each secret share into multiple plaintexts of a small enough size. Then, each participant ElGamal encrypts each chunk. Each party then generates a zero-knowledge proof that (1) each chunk falls within a reasonable range, and (2) the chunks correctly combine to the secret key share.

Let c be the number of chunks, and ℓ be the chunk size. The bandwidth overhead of ElGamal-DKG requires each party to send $(n - 1) \cdot c$ ElGamal ciphertexts (and one SNARK proof), and receive $c \cdot (n - 1)^2$ ElGamal ciphertexts (and $n - 1$ SNARK proofs). The computational overhead to each participant is dominated by performing $c \cdot (n - 1)$ ElGamal decryptions. As such, there is a fundamental tradeoff in bandwidth and computational overhead relative to the chunk size ℓ with ElGamal-DKG. On one end of the spectrum, for a security parameter of 128, setting $\ell = 1$ will result in each participant sending $128(n - 1)$ ElGamal ciphertexts, and receiving $128(n - 1)^2$ ElGamal ciphertexts in return. However, while increasing the chunk size ℓ in turn reduces each participant's bandwidth overhead, the computational cost for each participant increases, requiring (roughly) $c \cdot (n - 1) \cdot \sqrt{2}^\ell$ operations to perform ElGamal decryption, via Baby-step Giant-step [44].

Case Study Two: BHLS25-DKG, Private Channels, and eVRFs. Closest to Golden is prior work by Boneh et al. [12], who first define the notion of exponent verifiable random functions (eVRFs), and gives a DKG construction that uses exponent VRFs as a building block. We refer to this DKG as BHLS25-DKG. BHLS25-DKG achieves security with aborts, meaning that an adversary can force the protocol to terminate early and hence introduce bias into the output public key. In BHLS25-DKG, participants deterministically generate random coefficients for a secret polynomial using eVRFs, and then distribute Shamir secret shares to all other participants, which are points on this polynomial. Notably, participants distribute these shares over a private channel, and so avoid the use of public-key encryption. The authors claim that this construction requires only one round of communication, as all parties can verify the correctness of all other parties' commitments to their respective sharing polynomial, via proofs generated by

the eVRF. However, BHLS25-DKG in fact requires two rounds of interaction to protect against denial-of-service attacks, due to the use of a private channel to distribute shares. If any participant receives a malformed share, they must communicate this fact to all other participants. More specifically, an adversary may “split” the view of the honest participants, as it might follow the protocol honestly with some of the honest participants, and send invalid shares to the remaining honest participants. To prevent such an attack and to ensure that the output of the protocol is usable (i.e., to ensure that the secret key can in fact be recovered), all honest participants must perform an additional “consistency-checking” round (over a broadcast channel), indicating if the DKG completed successfully or not. As such, BHLS25-DKG requires two synchronous rounds of communication, because the scheme is not publicly verifiable.

2 Additional Related Work

Non-Interactive (One-Round) DKGs. We focus our discussion of related work to schemes which generate key material of the form $\text{sk} \in \mathbb{Z}_p, \text{PK} = g^{\text{sk}}$, where the output to each DKG participant is a Shamir secret share of sk .

Katz [54] gives a non-interactive DKG with two rounds of preprocessing, building on pseudorandom secret sharing by Cramer et al. [29] in the CRS model. However, the complexity of the scheme scales exponentially in n and t . More specifically, pseudorandom secret sharing by Cramer et al. requires each party to maintain additive secret shares with all subsets of possible signers, which requires the size of the secret key maintained by each party to be of size $\binom{n}{t}$.

Publicly Verifiable Secret Sharing (PVSS) is a common building block to non-interactive distributed key generation, where each party in the DKG acts as a Verifiable Secret Sharing (VSS) dealer, but generates in addition a proof or commitment to the secret sharing, such that anyone can verify the correctness of the share, without learning its value. A straightforward PVSS was defined by Schoenmakers [69], employing a variant of ElGamal public-key encryption and Chaum-Pedersen Discrete Logarithm Equality (DLEQ) proofs [23]. The resulting secret key $\text{sk}_k \in \mathbb{G}$ for each participant k is the group element $\text{sk}_k = g^{\sum_{i=1}^n f_i(k)}$, where $f_j(k) \in \mathbb{Z}_p$ is the Shamir secret share for party k issued by party j , and g is the generator for a prime-order group $\mathbb{G}$. While DKGs that build on this blueprint by Schoenmakers [20] sidestep the need for public-key decryption, the resulting secret key is an element of $\mathbb{G}$, and hence is incompatible for alternative parameter choices, such as discrete-logarithm schemes over elliptic curve groups, where secret keys are in the domain $\mathbb{Z}_p$.

Kate et al. [53] and Cascudo and David [19] define non-interactive DKGs from class groups, which employs a similar public-key encryption approach as Groth [48], but with improved decryption efficiency (as they leverage a subgroup where discrete log computation is easy). However, these schemes require additional security assumptions, whereas Golden requires only the discrete logarithm assumption and DDH.

Fouque and Stern [42] describe a one-round DKG that builds on the Paillier cryptosystem [64]. While this approach can be employed to generate discrete-logarithm key material, *Golden* offers a simpler approach by requiring only elliptic-curve groups.

Finally, we note there are recent batch optimizations to range proofs in the forms of *MissileProof* [45] and *DekartProof* [10], which may be applied to a scheme like Groth’s DKG so that the size of range proof for n receivers is constant or scales logarithmically instead of linearly. However, these proofs require a structured reference string (via trusted setup) to achieve the specified efficiency, which in practice requires an involved multi-party operation to generate and stronger trust assumptions [63, 74].

Synchronous Interactive (Multi-Round) DKGs. We next review a range of multi-round DKGs. However, we limit our review to synchronous DKGs, as the techniques employed in synchronous DKGs can be extended to the asynchronous settings. We refer to a systematization of knowledge by Bacho and Kavousi [3] for a review of asynchronous DKGs.

Pedersen-DKG [66] builds upon Feldman Verifiable Secret Sharing (VSS) [38], such that each party plays the role of the dealer. The protocol requires all parties to publicly broadcast their Feldman VSS polynomial commitment, and send each party their respective secret share over a private channel. At the end of the protocol, each party aggregates the secret shares received from all other parties, to determine their output secret key share. The public key is likewise the aggregation of the commitments to the secrets shared by each participant. To prevent a rogue-key attack, a zero-knowledge proof of knowledge of each party’s contribution to the DKG is required (as in the *PedPop* DKG defined by Komlo and Goldberg [56]). Furthermore, because the DKG does not allow for public verifiability, an additional consistency checking round is likewise required [68]. The protocol achieves security with aborts, and so does not protect against an adversary which forces the protocol to terminate early, thereby introducing bias into the output key.

Gurkan et al. [49] show that Pedersen-DKG can be used securely in any setting where the cryptographic scheme is “rekeyable,” such as in pairing-based groups. However, outside of such settings, it is possible for a malicious party to perform rogue-key attacks [6, 11].

Gennaro et al. [47] present a multi-round (unbiasable) DKG construction that ensures robustness against up to $t - 1$ players which may abort or deviate from the protocol. The DKG uses Pedersen commitments and verifiable secret sharing to distribute blinded commitments, and, in a subsequent round, opens the commitments after all participants have verified the integrity of their shares. The blinded commitments allow the scheme to ensure protection against a rushing adversary, which might otherwise perform a biasing attack, by waiting to receive contributions from all honest participants before selecting contributions for dishonest parties.

Cascudo et al. [21] define a multi-round (unbiasable) DKG that builds upon the ElGamal-based PVSS, similarly to Fouque and Stern [42]. However, to

avoid using Paillier encryption, the authors rely on a commit-and-prove strategy, where participants rely on the additively homomorphic property of ElGamal to demonstrate knowledge of their (aggregated) public key share.

Feng et al. [39] define a multi-round (unbiasable) DKG that builds upon the SCRAPE [20] technique for verifiable secret sharing, as well as ElGamal encryption.

Several synchronous DKGs have been introduced with adaptive security [1, 16, 39, 52], but require multiple rounds of interaction. For this work, we target static security. It is an interesting open question how our DKG can be proven secure against an adaptive adversary, following recent impossibility results for threshold signatures with public keys with a unique correspondence to secret keys (as is the case for keys generated by Golden) [31].

Bacho and Kavousi [3] give a systematization of knowledge for discrete-logarithm key generation schemes, comparing on differences such as number of rounds, synchronicity, and communication overhead. The authors focus on DKGs that satisfy the ideal functionality of Katz [54], for a fully secure DKG, which outputs key material that is indistinguishable from key material generation by a single-party key generation algorithm. However, our work targets one-round DKGs which cannot satisfy this ideal functionality [54], and instead must satisfy a slightly weaker notion, which we give in Figure 1.

3 Preliminaries

General Notation and Definitions. Let $\lambda \in \mathbb{N}$ denote the security parameter and 1^λ its unary representation. A function $f : \mathbb{N} \rightarrow \mathbb{R}$ is called *negligible* if for all $c \in \mathbb{R}, c > 0$, there exists $k_0 \in \mathbb{N}$ such that $|f(k)| < \frac{1}{k^c}$ for all $k \in \mathbb{N}, k \geq k_0$. For a non-empty set S , let $x \leftarrow_s S$ denote sampling an element of S uniformly at random and assigning it to x . We use $[n]$ to represent the set $\{1, \dots, n\}$ and $[0..n]$ to represent the set $\{0, \dots, n\}$.

Let PPT denote probabilistic polynomial time. Algorithms are randomized unless explicitly noted otherwise. Let $y \leftarrow A(x; \omega)$ denote running algorithm A on input x and randomness ω and assigning its output to y . Let $y \leftarrow_s A(x)$ denote $y \leftarrow A(x; \omega)$ for a uniformly random ω .

Group Generation. Let **GroupGen** be a polynomial-time algorithm that takes as input a security parameter 1^λ and outputs a group description $(E(\mathbb{F}_p), p, \mathbb{G}, s, g)$ consisting of an elliptic curve group $E(\mathbb{F}_p)$ over $\mathbb{F}_p$ for some prime p and a subgroup $\mathbb{G} \subset E(\mathbb{F}_p)$ of order s such that $|E(\mathbb{F}_p)| = h \cdot s$ where h is the cofactor; s is a λ -bit prime, and g is a generator of $\mathbb{G}$. This algorithm is used for PKI.

3.1 Distributed Key Generation (DKG) with Additive Bias

We follow the definition and idealized notion of security $\mathcal{F}_{\text{KeyGen}}^\Delta$ for a distributed key generation (DKG) protocol where the adversary is allowed to bias the output key with some (constrained) additive shift Δ , as presented by Katz [54, Figure 8];

This definition is weaker than full security for distributed key generation [47, 54], where the output public key is required to be from the same distribution as single-party key material. However, it is impossible to realize this stronger notion of security for asynchronous (one-round) DKGs, as proven by Katz [54]. However, the additive shift Δ is tolerable by some applications, such as threshold signatures [26, 30, 48], and so this weaker notion of security is relevant in practice, with the advantage of reducing network rounds of communication among parties.

We give a stand-alone definition of security for $\mathcal{F}_{\text{KeyGen}}^\Delta$. Because our proof for Golden uses only online straight-line simulation, our proof with respect to $\mathcal{F}_{\text{KeyGen}}^\Delta$ in the standalone setting is also by extension secure in the Universal Composability (UC) framework [57]. We assume an adversary that performs only static corruptions.

Let Π be an (n, t) -party DKG protocol, where n is the total number of parties and t is the threshold. A real-world execution of Π with respect to a PPT adversary $\mathcal{A}$ is as follows.

1. $\mathcal{A}$ specifies a set of corrupted parties $\text{corr} \subset [n]$, such that $|\text{corr}| < t$.
2. The honest parties participate in the protocol with the parties in corr . Honest parties follow the protocol, while corrupted parties are controlled by $\mathcal{A}$.
3. When an honest party terminates, it outputs a value as determined by the protocol description.
4. The view of $\mathcal{A}$ consists of 1) the randomness used by $\mathcal{A}$, 2) messages exchanged by honest and corrupt parties, and 3) messages sent over a broadcast channel.

Let $\text{Real}_{\Pi, \mathcal{A}}(1^\lambda)$ denote the random variable parameterized by the security parameter λ that is the tuple, consisting of 1) the corrupted parties corr , 2) the outputs of honest parties, and 3) the view of $\mathcal{A}$.

Now, let $\mathcal{F}$ be an (n, t) idealized (and randomized) functionality. An execution of $\mathcal{F}$ with respect to a PPT adversary $\mathcal{S}$ is as follows:

1. $\mathcal{S}$ specifies a set of corrupted parties $\text{corr} \subset [n]$, such that $|\text{corr}| < t$, and provides corr to $\mathcal{F}$.
2. When an honest party terminates, it outputs a value as determined by the protocol description.
3. $\mathcal{S}$ is allowed to instruct corrupted parties to interact with $\mathbb{F}$ however it likes.
4. Honest parties output the value(s) sent to them by $\mathbb{F}$.
5. $\mathcal{S}$ outputs an arbitrary function of its view.

Let $\text{Ideal}_{\mathcal{F}_{\text{KeyGen}}^\Delta, \mathcal{S}}(1^\lambda)$ denote the random variable parameterized by the security parameter λ that is the tuple, consisting of 1) the corrupted parties corr , 2) the outputs of honest parties, and 3) the view of $\mathcal{S}$.

Definition 1. A distributed key generation (DKG) protocol Π (n, t) -securely realizes an ideal functionality $\mathcal{F}$ if for every PPT real-world adversary $\mathcal{A}$, there exists a PPT ideal-world adversary $\mathcal{S}$, such that no efficient PPT distinguisher $\mathcal{D}$ can distinguish between $\text{Real}_{\Pi, \mathcal{A}}$ and $\text{Ideal}_{\mathcal{F}_{\text{KeyGen}}^\Delta, \mathcal{S}}$.

$$\mathcal{F}_{\text{KeyGen}}^\Delta$$

1. Let n, t be parameters provided to $\mathcal{F}_{\text{KeyGen}}^\Delta$.
2. Send (n, t) to the adversary $\mathcal{A}$; receive corr from $\mathcal{A}$.
3. Choose $x' \leftarrow \mathbb{Z}_p$ and compute $Y \leftarrow g^{x'}$.
4. Send Y to the adversary $\mathcal{A}$; receive $(\Delta, (\text{sk}_j)_{j \in \text{corr}})$ in return.
5. If $|\text{corr}| > 0$, set $x \leftarrow x' + \Delta$; else, set $x \leftarrow x'$.
6. Let f be the polynomial of degree t such that $f(0) = x$ and $f(j) = \text{sk}_j$, for $j \in \text{corr}$.
7. Set $\text{PK} = g^x$.
8. Set $\text{sk}_i = f(i)$, for $i \in \text{hon}$.
9. For $i \in [n]$, set $\text{PK}_i = g^{\text{sk}_i}$.
10. Send $(\text{PK}, \{\text{PK}_i\}_{i \in [n]}, \text{sk}_i)$ to each party $i \in [n]$; send $(\text{PK}, \{\text{PK}_i\}_{i \in [n]})$ to the adversary.

Fig. 1: Ideal functionality for key generation allowing additive bias, as defined by Katz [54].

Multi-Session Extension of $\mathcal{F}_{\text{KeyGen}}^\Delta$. As remarked by Katz [54], it is straightforward to verify that in the setting where parties maintain state after a setup phase, $\mathcal{F}_{\text{KeyGen}}^\Delta$ remains secure across repeated executions of the protocol, a requirement for UC security [17]. We similarly do not formalize this multi-session extension of $\mathcal{F}_{\text{KeyGen}}^\Delta$, but likewise remark $\mathcal{F}_{\text{KeyGen}}^\Delta$ can be extended to this setting.

3.2 Exponent Verifiable Random Functions

We employ the definition and notion of security of exponent verifiable random functions (eVRFs) as presented by Boneh et al. [12].

Definition 2. Let $\{(\mathcal{X}_\lambda, \mathcal{Y}_\lambda)\}$ be an ensemble of domains and ranges, and $\mathcal{Y}_\lambda$ be a finite cyclic group with generator g_λ . An **exponent verifiable random function (eVRF)** with respect to $\{(\mathcal{X}_\lambda, \mathcal{Y}_\lambda)\}$ is the tuple of algorithms (Setup, KeyGen, Evaluate, Verify), as follows:

- **Setup**(1^λ) $\rightarrow$ **par**: Accepts as input the security parameter λ , and outputs public parameters **par**, which are implicitly given as input to all other algorithms.
- **KeyGen**() $\rightarrow$ (**sk**, **PK**): Outputs a secret key **sk** and corresponding verification key **PK**.
- **Evaluate**(**sk**, x) $\rightarrow$ (α , A , π): Accepts as input a secret key **sk** and input to evaluate x , and outputs the evaluation output α , a commitment A to α , and proof π that the evaluation was performed correctly.
- **Verify**(**PK**, x , A , π): Accepts as input a public key **PK**, input to evaluate x , a commitment A to α , and proof π . Outputs 1 to indicate that A is a valid commitment to the eVRF input x .

$\text{Exp}_{\text{eVRF},b,\mathcal{A}}^{\text{psdr}}(\lambda)$	$\mathcal{O}^{\text{Evaluate}}(\text{sk}, x)$
1 : $\text{par} \leftarrow \$ \text{eVRF.Setup}(1^\lambda)$	1 : if $b = 0$
2 : $(\text{sk}, \text{PK}) \leftarrow \$ \text{eVRF.KeyGen}()$	2 : $(\alpha, A, \pi) \leftarrow \text{eVRF.Evaluate}(\text{sk}, x)$
3 : $b' \leftarrow \$ \mathcal{A}^{\mathcal{O}^{\text{Evaluate}}(\text{sk}, \cdot)}(\text{par}, \text{PK})$	3 : else // $b = 1$ case
4 : return b'	4 : $\alpha \leftarrow \$ \mathbb{F}_p$
	5 : // p is the order of $\mathbb{G}$
	6 : return α

Fig. 2: Pseudorandomness experiment defining the advantage of an adversary $\mathcal{A}$ against an exponent verifiable random function (eVRF) eVRF .

Game-Based Notions of Security An eVRF is secure if it is correct, pseudo-random, verifiable, and simulatable. We give these notions next.

Correctness. An eVRF is *correct* if for every PPT adversary $\mathcal{A}$ the following function $\text{Adv}_{\text{eVRF},\mathcal{A}}^{\text{corr}}(\lambda)$ is negligible:

$$\text{Adv}_{\text{eVRF},\mathcal{A}}^{\text{corr}}(\lambda) := \Pr \left[\begin{array}{l} \text{par} \leftarrow \$ \text{Setup}(1^\lambda) \\ (\text{sk}, \text{PK}) \leftarrow \$ \text{KeyGen}() \\ x \leftarrow \$ \mathcal{A}^{\text{Evaluate}(\text{sk}, \cdot)}(\text{par}, \text{PK}) \\ (\alpha, A, \pi) \leftarrow \text{Evaluate}(\text{sk}, x) \end{array} : \text{Verify}(\text{PK}, x, A, \pi) \neq 1 \right]$$

Pseudorandomness. An eVRF is *pseudorandom* if for every PPT adversary $\mathcal{A}$ the following function $\text{Adv}_{\text{eVRF},\mathcal{A}}^{\text{psdr}}(\lambda)$ is negligible (see Figure 2):

$$\text{Adv}_{\text{eVRF},\mathcal{A}}^{\text{psdr}}(\lambda) = \left| \Pr[\text{Exp}_{\text{eVRF},0,\mathcal{A}}^{\text{psdr}}(\lambda) = 1] - \Pr[\text{Exp}_{\text{eVRF},1,\mathcal{A}}^{\text{psdr}}(\lambda) = 1] \right|$$

Verifiability. An eVRF is *verifiable* if for every PPT adversary $\mathcal{A}$ the following function $\text{Adv}_{\text{eVRF},\mathcal{A}}^{\text{verf}}(\lambda)$ is negligible:

$$\text{Adv}_{\text{eVRF},\mathcal{A}}^{\text{verf}}(\lambda) = |\Pr[A_0 \neq A_1 \wedge \text{Verify}(\text{PK}, x, A_i, \pi_i) = 1 \ \forall i \in \{0, 1\}]|$$

where $\text{par} \leftarrow \$ \text{Setup}(1^\lambda)$ and $(\text{PK}, x, (A_0, \pi_0), (A_1, \pi_1)) \leftarrow \$ \mathcal{A}(\text{par})$.

Simulatability. An eVRF is *simulatable* if there exists a PPT algorithm SIM such that for every PPT distinguishing algorithm $\mathcal{D}$, for $\text{par} \leftarrow \$ \text{Setup}(1^\lambda)$ and $(\text{sk}, \text{PK}) \leftarrow \$ \text{KeyGen}()$, the following function $\text{Adv}_{\text{eVRF},\mathcal{D},\text{SIM}}^{\text{sim}}(\lambda)$ is negligible:

$$\text{Adv}_{\text{eVRF}, \mathcal{D}, \text{SIM}}^{\text{sim}}(\lambda) = \left| \Pr[\mathcal{D}^{\text{Evaluate}(\text{sk}, \cdot)}(\text{par}, \text{PK}) = 1] - \Pr[\mathcal{D}^{\text{SIM}(\text{PK}, \cdot)}(\text{par}, \text{PK}) = 1] \right|$$

Simulation-Based Notion of Security For our proof of simulation-based security for **Golden**, we rely on the simulation-based notion of security for an eVRF with respect to an idealized functionality $\mathcal{F}_{\text{eVRF}}$ as presented by Boneh et al. [12]. For reference, we review this functionality in Appendix A.

3.3 Polynomial Interpolation and Shamir Secret Sharing

A polynomial $f(x) = a_0 + a_1x + a_2x^2 + \dots + a_{t-1}x^{t-1}$ of degree $t-1$ over a field $\mathbb{F}$ can be interpolated by t points. Let $\eta \subseteq [n]$ be the list of t distinct indices corresponding to the x -coordinates $x_i \in \mathbb{F}$ of these points. Then the Lagrange polynomial $L_i(x)$ has the form:

$$L_i(x) = \prod_{j \in \eta; j \neq i} \frac{x - x_j}{x_i - x_j}$$

Given a set of t points $(x_i, f(x_i))_{i \in \eta}$, any point $f(\tilde{x})$ on the polynomial f can be determined by Lagrange interpolation as follows:

$$f(\tilde{x}) = \sum_{k \in \eta} f(x_k) \cdot L_k(\tilde{x}) .$$

Shamir Secret Sharing. Shamir secret sharing [71] is an (n, t) -threshold secret sharing scheme consisting of algorithms (**Share**, **Recover**), defined as follows:

- **Share** $(x, n, t) \rightarrow \{(1, \bar{x}_1), \dots, (n, \bar{x}_n)\}$: On input a secret x , number of participants n , and threshold t , perform the following. First, define a polynomial $f(Z) = x + a_1Z + a_2Z^2 + \dots + a_{t-1}Z^{t-1}$ with $a_1, \dots, a_{t-1} \leftarrow_{\$} \mathbb{Z}_p$. Then, set each participant's share $\bar{x}_i, i \in [n]$, to be the evaluation of $f(i)$:

$$\bar{x}_i \leftarrow x + \sum_{j \in [t-1]} a_j i^j .$$

Output $\{(i, \bar{x}_i)\}_{i \in [n]}$.

- **Recover** $(t, \{(i, \bar{x}_i)\}_{i \in C}) \rightarrow \perp/x$: On input threshold t and a set of shares $\{(i, \bar{x})\}_{i \in C}$, output $\perp$ if the coalition $C \not\subseteq [n]$ or if $|C| < t$. Otherwise, recover x as follows:

$$x \leftarrow \sum_{i \in C} \bar{x}_i \cdot L_i(0) ,$$

where the Lagrange coefficient for the set C is defined by

$$L_i(0) = \prod_{j \in C, j \neq i} \frac{j}{j - i} .$$

3.4 The Bulletproofs Argument System

The Bulletproofs argument system [15] can prove computation represented in rank-1 constraint system (R1CS) and is suitable when the statement is given in the exponent.

Consider the following R1CS relation:

$$\mathcal{R}_{R1CS} = \left\{ (A, B, C \in \mathbb{F}_p^{N \times M}, \theta \in \mathbb{G}_{\text{out}}) ; (\mathbf{x} \in \mathbb{F}_p^W, \mathbf{y} \in \mathbb{F}_p^{M-W}) \right\} \quad \text{where}$$

$$(1) \quad (A\mathbf{z}) \circ (B\mathbf{z}) = (C\mathbf{z}) \quad \text{where} \quad \mathbf{z} = (\mathbf{x}, \mathbf{y}) \in \mathbb{F}_p^M$$

$$(2) \quad \theta = \mathbf{g}^{\mathbf{x}}$$

The notation $u \circ v$ denotes the Hadamard product of vectors. The *prefix* $\mathbf{x} \in \mathbb{F}_p^W$ is provided as a Pedersen commitment, where $\mathbf{g}^{\mathbf{x}}$ denotes $\prod_{i=1}^W g_i^{x_i}$ for some vector $\mathbf{g}$ of generators in $\mathbb{G}_{\text{out}}$. Each row in the matrices A, B, C is called a *constraint*, and we refer to $\mathbf{z}$ as the *extended witness*.

Theorem 1 (Bulletproofs [14, 70]). *Bulletproofs is a zero-knowledge non-interactive argument of knowledge for the relation $\mathcal{R}_{R1CS}$ in the random oracle model, assuming the hardness of discrete logarithm in $\mathbb{G}_{\text{out}}$ for expected polynomial-time algorithms. The length of the proof is $2 \log_2(N + M) + 3$ group elements in $\mathbb{G}_{\text{out}}$ and 3 field elements.*

The Bulletproofs argument system is complete, computationally knowledge sound, and zero-knowledge. Computational knowledge soundness assumes the hardness of discrete logarithm and means that otherwise there is an expected polynomial-time algorithm that finds a nontrivial linear relation among the generators. The argument system can be made non-interactive using the Fiat-Shamir transform [40] in the random oracle model [2].

The length of the proof is $2 \log_2(N + M) + 3$ group elements in $\mathbb{G}_{\text{out}}$ and 3 field elements. The verifier complexity is dominated by a multi-scalar multiplication (MSM) of size $2(N + M)$, which can be done via Pippenger’s algorithm [67] as opposed to computing all exponentiations individually. Batching is also possible when verifying multiple proofs at once [15].

4 A New Exponent Verifiable Random Function

We now give a new eVRF construction, which we refer to as $\text{eVRF}^\dagger$. Our construction builds upon the DDH construction presented by Boneh et al. [12] in that our construction likewise makes use of the DDH PRF [61] $H_{\mathbb{G}_{\text{in}}}(\text{msg})^k$, where $H_{\mathbb{G}_{\text{in}}} : \{0, 1\}^* \rightarrow \mathbb{G}_{\text{in}}$ is modeled as a random oracle, and $\mathbb{G}_{\text{in}}$ is a cyclic subgroup of prime order s where DDH is assumed to be hard. However, our construction derives k as a shared value between two participants via a non-interactive key exchange (NIKE) performed in $\mathbb{G}_{\text{in}}$.

To derive the NIKE shared secret, our eVRF uses as a building block the Diffie-Hellman key exchange (denoted by DH). We define NIKE as a separate

Notation	Description
$E(\mathbb{F}_p)$	Elliptic curve defined over $\mathbb{F}_p$ for some prime p
$\mathbb{G}_{\text{in}}$	Subgroup of $E(\mathbb{F}_p)$ of prime order s
h	Cofactor such that $ E(\mathbb{F}_p) = h \cdot s$
$\mathbb{G}_{\text{out}}$	Subgroup of a different elliptic curve with the requirement that the order be p
$g_{\text{in}}, g_{\text{out}}$	Generators of $\mathbb{G}_{\text{in}}$ and $\mathbb{G}_{\text{out}}$, respectively
sk, PK	A registered key pair, where PK is a group element in $\mathbb{G}_{\text{in}}$
msg	Part of input to our eVRF
$\text{int}(\cdot)$	A function that casts an input to integer
$H_{\mathbb{G}_{\text{in}},1}, H_{\mathbb{G}_{\text{in}},2}$	Random oracles
β	Public field element in $\mathbb{F}_p$, a constant used by the leftover hash lemma

Table 3: Notation for $\text{eVRF}^\dagger$.

building block to make the proof of security more straightforward, and refer to Appendix B for the definition and security notions of a NIKE.

We present $\text{eVRF}^\dagger$ next; we refer to Table 3 for further notation used within our construction.

- $\text{Setup}(1^\lambda) \rightarrow \text{par}$: Generate public parameters $(E(\mathbb{F}_p), p, \mathbb{G}_{\text{in}}, s, g_{\text{in}})$ from $\text{DH.Setup}(1^\lambda)$; set $\text{par} \leftarrow (E(\mathbb{F}_p), p, \mathbb{G}_{\text{in}}, s, g_{\text{in}}, \mathbb{G}_{\text{out}}, g_{\text{out}}, H_{\mathbb{G}_{\text{in}},1}, H_{\mathbb{G}_{\text{in}},2}, \beta)$ where $\langle g_{\text{out}} \rangle = \mathbb{G}_{\text{out}}$ is a subgroup of a different elliptic curve with the requirement that the order be p , $\beta \leftarrow_s \mathbb{F}_p$ is used by the leftover hash lemma [50] described in Appendix C, and $H_{\mathbb{G}_{\text{in}},1}$ and $H_{\mathbb{G}_{\text{in}},2}$ are random oracles mapping from $\{0, 1\}^*$ to $\mathbb{G}_{\text{in}}$. Output par .
- $\text{KeyGen}() \rightarrow (\text{sk}, \text{PK})$: Derive $(\text{sk}, \text{PK}) \leftarrow \text{DH.KeyGen}()$, where PK is a group element in $\mathbb{G}_{\text{in}}$. Output (sk, PK) .
- $\text{Evaluate}(\text{sk}, x) \rightarrow (\alpha, A, \pi)$: Perform the following steps:
 1. Parse the input as $(\text{msg}, \text{PK}') \leftarrow x$.
 2. Derive $S \leftarrow \text{DH.GetSharedKey}(\text{sk}, \text{PK}')$.
 3. Derive $k_0 \leftarrow S.X$ where “ X ” retrieves the x -coordinate of S .
 4. Derive $k \leftarrow \text{int}(k_0)$ where $\text{int}(\cdot)$ is a function that casts an input to integer (so that we interpret k_0 as an integer in the form of k).
 5. Derive $\alpha \leftarrow \beta \cdot (H_{\mathbb{G}_{\text{in}},1}(\text{msg})^k \cdot X) + (H_{\mathbb{G}_{\text{in}},2}(\text{msg})^k \cdot X)$.
 6. Derive commitment $A \leftarrow g_{\text{out}}^\alpha$.
 7. Generate a zero-knowledge proof π for the relation $\mathcal{R}_{\text{eVRF}^\dagger}$ as given in Figure 3.
 8. Output (α, A, π) .
- $\text{Verify}(\text{PK}, x, A, \pi)$: Parse the input as $(\text{msg}, \text{PK}') \leftarrow x$. Then, verify the relation $\mathcal{R}_{\text{eVRF}^\dagger}$ as given in Figure 3, and output 1 if the relation holds; otherwise, output 0.

$$\mathcal{R}_{\text{eVRF}^\dagger} = \left\{ (\mathbb{G}_{\text{in}} \subset E(\mathbb{F}_p); \{H_{\mathbb{G}_{\text{in}},j}(\text{msg}), \text{PK}_j \in \mathbb{G}_{\text{in}}\}_{j=1}^2, R \in \mathbb{G}_{\text{out}}, \beta \in \mathbb{F}_p) ; (\text{sk}_1 \in \mathbb{F}_s) \right\}$$

iff

- (0) $\text{PK}_1 \stackrel{?}{=} g_{\text{in}}^{\text{sk}_1} \in \mathbb{G}_{\text{in}}$
- (1) $S \leftarrow \text{PK}_2^{\text{sk}_1} \in \mathbb{G}_{\text{in}}$
- (2) $k_0 \leftarrow S.X \in \mathbb{F}_p$
- (3) $k \leftarrow \text{int}(k_0) \in \mathbb{Z}$
- (4) $T_1 \leftarrow H_{\mathbb{G}_{\text{in}},1}(\text{msg})^k \in \mathbb{G}_{\text{in}}$
- (5) $T_2 \leftarrow H_{\mathbb{G}_{\text{in}},2}(\text{msg})^k \in \mathbb{G}_{\text{in}}$
- (6) $r_1 \leftarrow T_1.X \in \mathbb{F}_p$
- (7) $r_2 \leftarrow T_2.X \in \mathbb{F}_p$
- (8) $r \leftarrow \beta \cdot r_1 + r_2 \in \mathbb{F}_p$
- (9) $R \stackrel{?}{=} g_{\text{out}}^r \in \mathbb{G}_{\text{out}}$

Fig. 3: The relation $\mathcal{R}_{\text{eVRF}^\dagger}$ used in the new eVRF defined in Section 4. R is the eVRF commitment output. See Table 3 for additional information on notation.

We next demonstrate that $\text{eVRF}^\dagger$ is secure, i.e. that it is correct, pseudorandom, verifiable, and simulatable.

Theorem 2 (Security of $\text{eVRF}^\dagger$). *Let $(E(\mathbb{F}_p), p, \mathbb{G}_{\text{in}}, s, g_{\text{in}})$ be a tuple of public parameters for an explicit elliptic curve subgroup where DDH is hard. Then assuming a non-interactive zero-knowledge argument system which proves the relation $\mathcal{R}_{\text{eVRF}^\dagger}$, $\text{eVRF}^\dagger$ is a secure exponent verifiable random function.*

In particular, the adversary advantage in the pseudorandomness experiment is bounded from above as follows: given an adversary $\mathcal{A}$ against the eVRF pseudorandomness of $\text{eVRF}^\dagger$, there exists an adversary $\mathcal{B}_1$ against the session-key unrecoverability of the Diffie-Hellman non-interactive key exchange such that

$$\text{Adv}_{\text{eVRF}^\dagger, \mathcal{A}}^{\text{psdr}}(\lambda) \leq \text{Adv}_{\text{DH}, \mathcal{B}_1}^{\text{unrec}}(\lambda) + \frac{8h^2\sqrt{p}}{p-2h} = \text{Adv}_{\text{DH}, \mathcal{B}_1}^{\text{unrec}}(\lambda) + \mathcal{O}\left(\frac{1}{\sqrt{p}}\right)$$

where h is the cofactor and s is the prime order of the subgroup $\mathbb{G}_{\text{in}}$ of $E(\mathbb{F}_p)$ satisfying $|E(\mathbb{F}_p)| = h \cdot s$.

We give the proof of Theorem 2 in Appendix D. The main idea is that we make a reduction to the session-key unrecoverability (Appendix B) of a NIKE, the Diffie-Hellman key exchange in our case, and then proceed to show pseudorandomness by applying the leftover hash lemma [50] (Appendix C) in light of our construction. Our analysis is valid even for elliptic curves with a nontrivial cofactor, which is a practical merit not considered in prior work.

4.1 Argument System for the Relation $\mathcal{R}_{\text{eVRF}^\dagger}$

While a generic zkSNARK (zero-knowledge succinct non-interactive argument of knowledge) system [8] could be used to prove $\mathcal{R}_{\text{eVRF}^\dagger}$, we prove it using Bulletproofs [15] (see Section 3.4).

More precisely, we arithmetize $\mathcal{R}_{\text{eVRF}^\dagger}$ except the last step (9) (from Figure 3), let this arithmetization in R1CS define the A, B, C matrices for our Bulletproofs argument system, and furthermore combine the required Pedersen commitment $\theta := g_{\text{out},1} \cdot g_{\text{out}}^r$ (such that our prefix is $(1, r)$ and $g_{\text{out},1}$ is another independent generator) with a proof of knowledge of discrete logarithm of g_{out}^r to effectively prove $\mathcal{R}_{\text{eVRF}^\dagger}$ as a whole with $R = g_{\text{out}}^r$. Note that this is different from naively turning $\mathcal{R}_{\text{eVRF}^\dagger}$ itself into A, B, C matrices for Bulletproofs, which emits different properties as remarked later.

At a high level, we need to efficiently represent an exponentiation of a group element (of an elliptic curve group) as R1CS constraints. Attributed to Boneh et al. [12], the techniques we leverage to prove computation of $\tilde{g}^k$ for some arbitrary k (private) and group element $\tilde{g}$ (public) involve a two-step process. First, we bit-decompose k into $(k_0, k_1, \dots, k_\lambda) \in \{0, 1\}^{\lambda+1}$ such that $k = \sum_{i=0}^{\lambda} 2^i \cdot k_i$. Second, we incrementally construct $\tilde{g}^k$ using $(k_0, k_1, \dots, k_\lambda)$ and a set of precomputed points such that all incremental constructions can be represented as R1CS constraints. To facilitate analysis, we refer to each of these two steps as a “gadget” and thus are able to offer a modular presentation in discussing the four required exponentiations $g_{\text{in}}^{\text{sk}_1}$, $\text{PK}_2^{\text{sk}_1}$, $\text{H}_{\mathbb{G}_{\text{in}},1}(\text{msg})^k$, and $\text{H}_{\mathbb{G}_{\text{in}},2}(\text{msg})^k$.

The end result for our prover is that our constraint system has $14\lambda + 14$ (= 3598 for $\lambda = 256$) constraints. We delineate the two gadgets and the derivation of the overall number of constraints below.

4.2 First Gadget: Bit-Decomposition Check

Checking that k bit-decomposes into $(k_0, k_1, \dots, k_\lambda) \in \{0, 1\}^{\lambda+1}$ in the form of R1CS constraints is relatively easier. The constraints are given by

- $k_i(k_i - 1) = 0$ for $i = 0, \dots, \lambda$
- $k = \sum_{i=0}^{\lambda} 2^i \cdot k_i$

totaling $\lambda + 2$ constraints. The extended witness variables are $k_0, k_1, \dots, k_\lambda, k$.

4.3 Second Gadget: Exponentiation Check

Given the bit-decomposition $(k_0, k_1, \dots, k_\lambda)$ of k , it is possible to efficiently represent the computation of $\tilde{g}^k$ (for some public group element $\tilde{g}$) as R1CS constraints as follows. We incrementally construct $\tilde{g}^k$ via a sequence of $\lambda + 1$ points $\{L_i\}_{i=0}^{\lambda}$ where $L_\lambda = \tilde{g}^k$. Specifically, we define $L_i := \prod_{j=0}^i \Delta_j$ where:

- $\Delta_j := P_j^{k_j} \cdot C_j$
- $P_j := \tilde{g}^{2^j}$

- $C_j := g^{c_j}$
- $c_j := \begin{cases} 1 & j = 0 \\ 2 & j = 1, \dots, \lambda - 1 \\ s - 2\lambda - 1 & j = \lambda \end{cases}$

Note P_j , C_j , and c_j can be publicly computed. The explanation is as follows. Without C_j , it is easy to see that $\tilde{g}^k = \prod_{j=0}^{\lambda} P_j^{k_j}$ simply from the bit-decomposition of k . Likewise, $\tilde{g}^k = \prod_{j=0}^{\lambda} P_j^{k_j} \cdot C_j = \prod_{j=0}^{\lambda} \Delta_j = L_{\lambda}$ from the fact that $\prod_{j=0}^{\lambda} C_j = 1$. In fact, the c_j values are deliberately chosen to satisfy this. More importantly, the c_j values (hence the C_j values) are deliberately chosen to offer a sort of a “correction” term to each $P_j^{k_j}$ so that each resulting incremental computation of $L_i = L_{i-1} \cdot \Delta_i$ is guaranteed to be a group operation involving two finite points in the elliptic curve group that have distinct x -coordinates, except with negligible probability (as shown by Boneh et al. [12, Lemma 6]). As a result, only the chord rule (as opposed to the tangent rule) of the elliptic curve group operation formula is used to calculate $\{L_i\}_{i=0}^{\lambda}$.

This means that the incremental computation starting from L_0 and ending with $L_{\lambda} = \tilde{g}^k$ uses the same formula and thus can be represented in R1CS in a consistent manner. Specifically, we achieve this using 4 intermediate (i.e. part of the extended witness) variables $(k_i, s_i, x_{L_i}, y_{L_i}) \in \mathbb{F}_p^4$ introduced in each iteration of computing $L_i = L_{i-1} \cdot \Delta_i$, where:

- k_i is from the bit-decomposition of k
- $s_i = (y_{L_{i-1}} - y_{\Delta_i}) \cdot (x_{L_{i-1}} - x_{\Delta_i})^{-1}$ is the slope of the line (which is needed to compute L_i) going through L_{i-1} and Δ_i
- x_{L_i} is the x -coordinate of L_i
- y_{L_i} is the y -coordinate of L_i

The needed constraints (for $i = 1, \dots, \lambda$) are as follows, where:

- $s_i \cdot (x_{L_{i-1}} - x_{\Delta_i}) = y_{L_{i-1}} - y_{\Delta_i}$ shows computation of the slope s_i
- $s_i \cdot s_i = x_{L_{i-1}} + x_{L_i} + x_{\Delta_i}$ shows computation of x_{L_i}
- $s_i \cdot (x_{L_{i-1}} - x_{L_i}) = y_{L_{i-1}} + y_{L_i}$ shows computation of y_{L_i}

We note that x_{Δ_i} and y_{Δ_i} are actually public linear functions of k_i (as Δ_i is either C_i or $P_i \cdot C_i$ while P_i and C_i are public), so there is no need to include them as part of the extended witness (though helpful for the ease of presentation above). Respectively, x_{Δ_i} and y_{Δ_i} can be substituted by $(x_{P_i \cdot C_i} - x_{C_i}) \cdot k_i + x_{C_i}$ and $(y_{P_i \cdot C_i} - y_{C_i}) \cdot k_i + y_{C_i}$ in practice. In addition, we do need to count the base case when $i = 0$ to show computation of x_{L_0} and y_{L_0} (which are equal to x_{Δ_0} and y_{Δ_0} by definition and thus are also linear functions of k_0), as they do not involve the above slope computation; this incurs two additional constraints.

In sum, this gadget incurs $3\lambda + 2$ constraints, and the extended witness variables are $k_0, x_{L_0}, y_{L_0}, k_1, s_1, x_{L_1}, y_{L_1}, \dots, k_{\lambda}, s_{\lambda}, x_{L_{\lambda}},$ and $y_{L_{\lambda}}$.

4.4 No Need to Arithmetize the Last Exponentiation

While the first four exponentiations $g_{\text{in}}^{\text{sk}_1}$, $\text{PK}_2^{\text{sk}_1}$, $H_{\mathbb{G}_{\text{in}},1}(\text{msg})^k$, and $H_{\mathbb{G}_{\text{in}},2}(\text{msg})^k$ are indeed represented as R1CS constraints (using the above gadgets), we can omit this process for $\mathcal{R}_{\text{eVRF}\dagger}$'s last exponentiation g_{out}^r and thus save constraints. The reason is that the eVRF commitment output $R = g_{\text{out}}^r$ and the Bulletproofs Pedersen commitment $\theta := g_{\text{out},1} \cdot g_{\text{out}}^r$ are in fact related in form such that providing g_{out}^r and its proof of knowledge of discrete logarithm essentially achieves the same effect as providing θ and the correctness of computation of g_{out}^r . The verifier can check this proof of knowledge to verify the ninth step of $\mathcal{R}_{\text{eVRF}\dagger}$.

4.5 Combining Everything for $\mathcal{R}_{\text{eVRF}\dagger}$

Now, proving $\mathcal{R}_{\text{eVRF}\dagger}$ involves 2 iterations of the first bit-decomposition gadget and 4 iterations of the second exponentiation gadget. Namely, steps (0) through (3) of $\mathcal{R}_{\text{eVRF}\dagger}$ (from Figure 3) show $g_{\text{in}}^{\text{sk}_1}$ and $\text{PK}_2^{\text{sk}_1}$ by using the first gadget once to show bit-decomposition of sk_1 and the second gadget twice to respectively show exponentiation of g_{in} and PK_2 by sk_1 . One additional constraint is used in step (0) to check equality, which is how the predetermined PK_1 is tied into $\mathcal{R}_{\text{eVRF}\dagger}$. Steps (4) through (7) show $H_{\mathbb{G}_{\text{in}},1}(\text{msg})^k$ and $H_{\mathbb{G}_{\text{in}},2}(\text{msg})^k$; the first gadget is used once to show bit-decomposition of k , and then the same bit-decomposition is used by two separate iterations of the second gadget, which respectively show exponentiation of $H_{\mathbb{G}_{\text{in}},1}(\text{msg})$ and $H_{\mathbb{G}_{\text{in}},2}(\text{msg})$ by k .

The eighth step of $\mathcal{R}_{\text{eVRF}\dagger}$ ($r \leftarrow \beta \cdot r_1 + r_2 \in \mathbb{F}_p$) is simply one constraint and is where the leftover hash lemma is invoked. The ninth step of $\mathcal{R}_{\text{eVRF}\dagger}$ is shown outside the Bulletproofs argument system by $g_{\text{out}}^r = R$ and its proof of knowledge of discrete logarithm as noted above. The summary is that our prover for $\mathcal{R}_{\text{eVRF}\dagger}$ computes Bulletproofs for a constraint system with $2 \cdot (\lambda + 2) + 4 \cdot (3\lambda + 2) + 2 = 14\lambda + 14$ ($= 3598$ for $\lambda = 256$) constraints and $14\lambda + 14$ extended witness variables, plus one proof of knowledge of discrete logarithm.

Remark A different proof construction is possible if we instead represent a slight variant of $\mathcal{R}_{\text{eVRF}\dagger}$ as A, B, C matrices for Bulletproofs, where the PKI group and the DKG group can be made equal (i.e. $\mathbb{G}_{\text{in}} = \mathbb{G}_{\text{out}}$). See Appendix E.

Remark A recent proof technique also mentioned by Boneh et al. [12], Eagen's approach [4, 36, 65] involving concepts from algebraic geometry such as principal divisors and Weil reciprocity has the effect of reducing the number of constraints for our second gadget (Section 4.3) from $3\lambda + 2$ to 7. Applied in our setting, such optimization would bring down the total number of constraints from $14\lambda + 14$ to $2 \cdot (\lambda + 2) + 4 \cdot (7) + 2 = 2\lambda + 34 = 546$ for $\lambda = 256$. The reduction is about 85%.

4.6 Concrete Efficiency, Batch Proving, and Batch Verification

We are using the R1CS instantiation of Bulletproofs [14, 70]. The R1CS constraint system for $\mathcal{R}_{\text{eVRF}\dagger}$ uses matrices of dimension $N = M = 3598$. We estimate the

Statements ($\tilde{n}$)	Constraints	Prover	Verifier	$ \pi $	$\tilde{n}$ proofs	Batch Verification
1	3598	0.3s	0.1s	1.5kb	1.5kb	0.1s
9	24158	1.8s	0.5s	1.9kb	17kb	0.6s
49	126958	6.8s	2.5s	2.1kb	101kb	6.7s
99	255458	13.5s	4.8s	2.2kb	214kb	22.1s

Table 4: Our eVRF performance estimates per zkalc [37] for BLS12-381. Statements measure the number of $\mathcal{R}_{\text{eVRF}^\dagger}$ statements that are proven. Constraints refer to the total number of R1CS constraints (which is also the R1CS extended witness size). Batch verification measures the time required to verify $\tilde{n}$ proofs simultaneously (as in the DKG application).

performance of the protocol implemented in arkworks [28], via zkalc [37] using curve BLS12-381 on AWS EC2 m5.2xlarge. We will provide an implementation for the full version, but estimate that it is within a factor of 2 of zkalc’s estimate.

In our DKG application, each client sends one proof to each other participant, i.e., creates $\tilde{n}$ proofs for $\tilde{n} + 1$ participants. For additional efficiency, the prover can alternatively create one proof for the conjunction of $\tilde{n}$ statements. This can dramatically improve the proof size, as it scales only logarithmically with the size of the statement. It can also improve the prover and verifier time, as multi-scalar multiplications (MSMs), the most expensive operation, scale as $\lambda \cdot N / \log N$ for MSMs of size N [67].

Finally, even with batch proving, each client will receive and verify $\tilde{n}$ proofs, one per other client. Bulletproofs enables efficient batch verification [15], where the most expensive verification operation (MSM) can be shared.

Prover efficiency For an R1CS instance with dimensions N and M , the prover’s computation is dominated by computing a Pedersen commitment of length $N + 2M$ and a zero-knowledge inner product argument [27, 70] with vectors of length $N + M$. Let $\rho = 2^{\lceil \log_2(N+M) \rceil}$. In the Bulletproofs inner product argument, the prover computes 2 MSMs of size $\rho, \rho/2, \rho/4, \dots, 1$. In our setting, $N = M = 3598$ and $\rho = 2^{13}$. The prover is therefore dominated by an MSM of size 10794 and two MSMs of size 8192. The field operations are $O(N + M)$ and hardly contribute to the prover time. Including all group and field operations, the zkalc-estimated prover runtime is 305 ms. For larger $\tilde{n}$, the constraint system for $\mathcal{R}_{\text{eVRF}^\dagger}^{\tilde{n}}$ requires $\tilde{n} \cdot 2570 + 1028$ constraints (bit-decomposing sk_1 and the $g_{\text{in}}^{\text{sk}_1}$ exponentiation can be re-used). We give estimated constraint counts and prover runtimes in Table 4.

Verifier efficiency The verifier needs to verify an inner product argument. The cost of this is dominated by an MSM of size $2(N + M) + 2\lceil \log_2(N + M) \rceil + 3$. The verifier also computes $O(\log N)$ hashes and $O(N + M)$ field operations, all of which add less than 5 ms to the runtime and is negligible compared to the MSM. For $N = M = 3598$, zkalc estimates a verifier time of 809 ms. In the DKG application,

the verifier is run by the receiver and will verify $\tilde{n}$ proofs for $\mathcal{R}_{\text{eVRF}^\dagger}^{\tilde{n}}$, assuming $\tilde{n}+1$ clients. The verifier can use the batch verification technique from Bulletproofs [15]. Using this, the MSM cost only scales as $2(N+M) + \tilde{n} \cdot 2\lceil \log_2(N+M) \rceil$. The scalar multiplications are upper bounded by $16 \cdot \tilde{n} \cdot (N+M)$. We give concrete estimates for $\tilde{n} = 1, 9, 49, 99$ in Table 4.

Proof size From Theorem 1, the proof size for one statement is $2\log_2(N+M)+3 \approx 29$ group elements and 3 field elements for $\lambda = 256$, plus $|\pi_{DL}|$ for some suitable proof of knowledge of discrete logarithm π_{DL} . In the batch proof setting, the proof size only scales logarithmically in $\tilde{n}$, e.g. for $\tilde{n} = 99$, the proof size is still only 43 group elements or 2.2 kb when using BLS12-381, which has 48-byte group elements. We present the proof size for different statement sizes in Table 4, and also present the size of $\tilde{n}$ proofs in light of the DKG context.

5 Golden: Lightweight Non-Interactive Distributed Key Generation

We next introduce our non-interactive DKG Golden. At a high level, Golden builds upon the eVRF $\text{eVRF}^\dagger$ presented in Section 4, as well as Shamir secret sharing as described in Section 3.3. We additionally require a PKI, as described next.

5.1 Requirement of a Discrete-Log Based PKI

We assume the use of a PKI where each party i maintains a discrete-logarithm keypair $(\text{sk}_i^I, \text{PK}_i^I)$ such that $\text{PK}_i^I = g^{\text{sk}_i^I} \in \mathbb{G}_{\text{in}}$. Furthermore, we additionally require that when registering public keys with the PKI, the PKI ensures that participant i proves knowledge of the corresponding private key sk_i^I to their public identity key PK_i^I . We describe this PKI as an ideal functionality $\mathcal{F}_{\text{pki}}$ in Appendix F, and give a concrete protocol in Appendix F.2. Finally, we require that all parties maintain a consistent view of the public keys and identities $\{(j, \text{PK}_j^I)\}_{j \in [n]}$.

5.2 Protocol Description

Each participant $i \in [n]$ in the DKG is initialized with their own identity key $(\text{sk}_i^I, \text{PK}_i^I)$, as well as the public identity keys of all participants $\{\text{PK}_j^I\}_{j \in [n]}$. Because we assume a PKI, these keys are assumed to be valid.

In the first round of the DKG, each participant i begins by sampling a random secret $\omega_i \leftarrow \mathbb{Z}_p$, and then performs Shamir secret sharing to obtain the shares $\{\bar{x}_{i,j}\}_{j=1}^n$ and the commitment $\bar{C}_i = (A_{i,0}, \dots, A_{i,t-1})$. Here, $\bar{C}_i$ is a vector commitment which commits to a random polynomial f_i such that $\bar{x}_{i,j} = f_i(j)$ and $f_i(0) = \omega_i$.

Each participant i then samples a random message $\text{msg}_i \leftarrow \{0, 1\}^\lambda$. Next, for all participants $j \in [n], j \neq i$, each participant i does the following. They first generate $(r_{i,j}, R_{i,j}, \pi_{i,j})$ by performing $\text{eVRF}^\dagger.\text{Evaluate}(\text{sk}_i^I, (\text{msg}_i, \text{PK}_j^I))$. Then

$\text{Round}_0(n, t, i, \text{sk}_i^I, \{(j, \text{PK}_j^I)\}_{j \in [n]}) \quad // \text{ Party } i \text{ performs the protocol.}$	
1 :	$\omega_i \leftarrow \mathbb{Z}_p$
2 :	$(\{\bar{x}_{i,j}\}_{j=1}^n, \bar{C}_i = (A_{i,0}, \dots, A_{i,t-1})) \leftarrow \text{Shamir.Share}(\omega_i, n, t)$
3 :	$\text{msg}_i \leftarrow \{0, 1\}^\lambda \quad // \text{ Sample a random message}$
4 :	for $j \in [n], j \neq i$ do
5 :	$(r_{i,j}, R_{i,j}, \pi_{i,j}) \leftarrow \text{eVRF}^\dagger.\text{Evaluate}(\text{sk}_i^I, (\text{msg}_i, \text{PK}_j^I))$
6 :	$z_{i,j} \leftarrow r_{i,j} + \bar{x}_{i,j} \quad // \text{ Encrypt the share to player } j$
7 :	$\sigma_{i,j} \leftarrow (R_{i,j}, z_{i,j})$
8 :	$\text{st}_i \leftarrow \bar{x}_{i,i} \quad // \text{ Keep one share for itself}$
9 :	$\text{bmsg}_i \leftarrow \{(\text{msg}_i, \bar{C}_i, \sigma_{i,j}, \pi_{i,j})\}_{j \in [n], j \neq i}$
10 :	Broadcast bmsg_i
$\text{Round}_1(n, t, i, \text{sk}_i^I, \{(j, \text{PK}_j^I)\}_{j \in [n]}, \{\text{bmsg}_j\}_{j \in [n]}, \text{st}_i)$	
1 :	$\bar{x}_{i,i} \leftarrow \text{st}_i$
2 :	for $j \in [n], j \neq i$, do
3 :	Parse $\{(\text{msg}_j, \bar{C}_j, \sigma_{j,k}, \pi_{j,k})\}_{k \in [n], k \neq j} \leftarrow \text{bmsg}_j$
4 :	Parse $(A_{j,0}, \dots, A_{j,t-1}) \leftarrow \bar{C}_j$; $(R_{j,k}, z_{j,k}) \leftarrow \sigma_{j,k}, k \in [n], k \neq j$
5 :	for $j \in [n], j \neq i$, do
6 :	for $k \in [n], k \neq j$, do
7 :	return $\perp$ if $\text{eVRF}^\dagger.\text{Verify}(\text{PK}_j^I, (\text{msg}_j, \text{PK}_k^I), R_{j,k}, \pi_{j,k}) \neq 1$
8 :	$\bar{X}_{j,k} \leftarrow \prod_{\ell=0}^{t-1} A_{j,\ell}^{k^\ell} \quad // \bar{X}_{j,k} \text{ is the commitment } g^{f_j(k)}$
9 :	return $\perp$ if $g^{z_{j,k}} \neq R_{j,k} \cdot \bar{X}_{j,k} \quad // \text{ Check the ciphertext is valid}$
10 :	for $j \in [n], j \neq i$, do
11 :	$(r_{j,i}, \cdot, \cdot) \leftarrow \text{eVRF}^\dagger.\text{Evaluate}(\text{sk}_i^I, (\text{msg}_j, \text{PK}_j^I)) \quad // \text{ Does not require } (R_{j,i}, \pi_{j,i})$
12 :	$\bar{x}_{j,i} \leftarrow z_{j,i} - r_{j,i} \quad // \text{ Decrypt Shamir share from participant } j$
13 :	$\text{sk}_i \leftarrow \sum_{j=1}^n \bar{x}_{j,i}$
14 :	for $\ell \in [n]$ do
15 :	$\text{PK}_\ell \leftarrow \prod_{k=1}^n \bar{X}_{k,\ell}$
16 :	$\text{PK} \leftarrow \prod_{k=1}^n A_{k,0} \quad // \text{ Equal to } g^{\sum_{k=1}^n \omega_k}$
17 :	return $(\text{PK}, \{\text{PK}_j\}_{j=1}^n, \text{sk}_i)$

Fig. 4: Golden, a non-interactive distributed key generation protocol that requires only one round of interaction. PK^I denotes identity public keys established via a PKI, as described in Appendix F. $\text{eVRF}^\dagger$ is defined in Section 4, and Shamir in Section 3.3.

they generate the ciphertext $z_{i,j}$ by performing $z_{i,j} \leftarrow r_{i,j} + \bar{x}_{i,j}$, encrypted using the nonce $r_{i,j}$ output from the eVRF. They finally set $\sigma_{i,j} \leftarrow (R_{i,j}, z_{i,j})$, where $R_{i,j} = g^{r_{i,j}}$ is also determined by the eVRF. Each participant finally sets $\text{bmsg}_i \leftarrow \{(\text{msg}_i, \bar{C}_i, \sigma_{i,j}, \pi_{i,j})\}_{j \in [n], j \neq i}$ to be a broadcast message, and broadcasts bmsg_i to all other participants.

After receiving and parsing bmsg_j , for $j \in [n], j \neq i$, each participant i verifies that the received eVRF proofs are valid for all participants $k \in [n], k \neq j$, by checking the relation $\text{eVRF}^\dagger.\text{Verify}(\text{PK}_j^I, (\text{msg}_j, \text{PK}_k^I), R_{j,k}, \pi_{j,k}) = 1$, if the check does not hold, the participant aborts. Because each bmsg_j is sent over a broadcast channel, if this check fails for any one participant, it will likewise fail for others.

Next, each participant derives $\bar{X}_{j,k} \leftarrow \prod_{\ell=0}^{t-1} A_{j,\ell}^{k^\ell}$, where $\bar{X}_{j,k} = g^{\bar{x}_{j,k}} = g^{f_j(k)}$ if the protocol was followed correctly. They then check the validity of the ciphertext, by checking the relation $g^{z_{j,k}} \stackrel{?}{=} R_{j,k} \cdot \bar{X}_{j,k}$, aborting if the check fails. Again, because all values are sent over broadcast, if any participant aborts, all other participants will abort.

Next, each participant i will decrypt their shares, by first deriving $(r_{j,i}, \cdot, \cdot) \leftarrow \text{eVRF}^\dagger.\text{Evaluate}(\text{sk}_i^I, (\text{msg}_j, \text{PK}_j^I))$. Note that unlike in the first round of the protocol, this step does not require the commitment $R_{j,i}$ or the proof $\pi_{j,i}$, and so can be performed more efficiently. Then, each participant will decrypt their Shamir share from each participant $j \neq i$ by performing $\bar{x}_{j,i} \leftarrow z_{j,i} - r_{j,i}$.

Finally, each participant obtains their secret key share sk_i by deriving $\text{sk}_i \leftarrow \sum_{j=1}^n \bar{x}_{j,i}$ using the shares sent from all parties. Additionally, participants derive the public key shares $\text{PK}_i \in \mathbb{G}_{\text{out}}$ for all participants $i \in [n]$, using the commitments $\bar{X}_{j,k} = g^{\bar{x}_{j,k}} = g^{f_j(k)}$. Participants derive the public key using the commitments $A_{i,0}$ via $\text{PK} = \prod_{i=1}^n A_{i,0} = g^{\sum_{i=1}^n f_i(0)} = g^{\sum_{i=1}^n \omega_i} \in \mathbb{G}_{\text{out}}$.

Correctness. Golden is correct assuming $\text{eVRF}^\dagger$ is correct and Shamir secret sharing is correct.

Firstly, when party i generates $(r_{i,j}, R_{i,j}, \pi_{i,j}) \leftarrow \text{eVRF}^\dagger.\text{Evaluate}(\text{sk}_i^I, (\text{msg}_i, \text{PK}_j^I))$ for party j , if $\text{eVRF}^\dagger$ is correct, then party j will obtain $(r'_{j,i}, R'_{j,i}, \pi'_{j,i}) \leftarrow \text{eVRF}^\dagger.\text{Evaluate}(\text{sk}_j^I, (\text{msg}_i, \text{PK}_i^I))$ with msg_i from party i , such that $r'_{j,i} = r_{i,j}$.

For this reason, after deriving $r_{j,i}$, each party i will recover the correct share $\bar{x}_{j,i} \leftarrow z_{j,i} - r_{j,i}$. Recall that each received secret share $\bar{x}_{j,k}, k \in [n]$ is a Shamir secret share of party j 's secret ω_j , such that $\omega_j = f_j(0)$ and each share $\bar{x}_{j,k} = f_j(k)$, where f_j is a random degree $t-1$ polynomial defined by the Shamir secret sharing algorithm Shamir.Share . As such, after each party derives their secret key share sk_i via $\text{sk}_i \leftarrow \sum_{j=1}^n \bar{x}_{j,i}$, the resulting secret key sk is the following: $\text{sk} = \sum_{i \in C} \text{sk}_i \lambda_i$, for all $C \subseteq [n]$, such that $|C| \geq t$. As such, $\text{sk} = \sum_{i \in C} \text{sk}_i \lambda_i = \sum_{i \in C} \sum_{j=1}^n \bar{x}_{j,i} \lambda_i$ and so by polynomial interpolation of Shamir secret sharing, $\text{sk} = \sum_{i=1}^n \omega_i$. Furthermore, $\text{PK} = \prod \text{PK}_i = g^{\text{sk}}$, because $\text{PK} \leftarrow \prod_{k=1}^n A_{k,0} = g^{\sum_{i=1}^n \omega_i} = g^{\text{sk}}$. Similarly, each participant's public key PK_i is likewise derived by performing "interpolation in the exponent," via $\text{PK}_i \leftarrow \prod_{k=1}^n \prod_{j=0}^{t-1} A_{k,j}^{i^j} = g^{\sum_{j=1}^n f_j(i)} = g^{\text{sk}_i}$.

Key Resharing We note that similarly to other distributed key generation schemes, **Golden** can be adapted to support distributed key resharing, where participants rotate their secret shares sk_i , but where the group keys sk and PK remain the same. The common technique to doing so is using zero secret sharing, where participants perform the protocol as defined in Figure 4, with the following changes. First, instead of sampling ω_i at random, each participant instead sets $\omega_i = 0$. Second, each participant will perform one additional check in **Round**₁, by checking that $A_{j,0}$ equals the identity of the group, for all $j \in [n], j \neq i$ (thereby checking that $f_j(0) = 0$).

5.3 Optimized Golden via Batching

We next review an optimized variant of **Golden**, which uses batching techniques to minimize communication and computational overhead. We present **Golden** in Figure 4 without these optimizations for simplicity and ease of understanding. We calculate our performance estimates with respect to the curve BLS12-381, for closest comparison to the Groth DKG implementation [48, 53].

Batch Proving and Verification. As aforementioned, batch proving allows each participant to generate one (as opposed to $n - 1$) proof showing correctness of $n - 1$ eVRFs. Hence, each participant verifies $n - 1$ proofs from others as opposed to $(n - 1)^2$ proofs. Below is a summary of the optimizations.

1. While the statement size of the proof naturally scales linearly, the size of the proof itself scales logarithmically in n , e.g. for 100 participants instead of generating 99 proofs of size 1.5 kb each, a prover sends only one proof of size 2.2 kb. For a DKG, the communication overhead is in fact dominated by each participant sending and receiving the $\bar{C}_i$ values, not by eVRF proofs.
2. The constant for the linear scaling of the proof’s statement size (number of constraints) actually reduces by about 29%. As noted before, the reason is that some of the gadgets can be re-used for each additional $\mathcal{R}_{\text{eVRF}^\dagger}$ statement due to the fact that a prover uses the same PKI key pair to compute $n - 1$ eVRF evaluations. Namely, $4\lambda + 4$ constraints out of $14\lambda + 14$ can be re-used.
3. Batch proving also benefits from computation time that is in fact slightly sublinear (by a logarithmic factor) in the statement size [67].

Even with batch proving, each participant still needs to verify $n - 1$ proofs. To further optimize this, we employ batch verification from Bulletproofs [15]. It enables sharing the most expensive operation (MSM) across all $n - 1$ proofs. For instance, checking 1 proof (for 99 eVRFs) might take 4.8 seconds, but checking 99 proofs only takes 22.1 seconds.

In Table 5, we present the overall optimized and unoptimized runtime and communication complexity of **Golden** for different numbers of participants.

6 Security

We prove security for **Golden** as presented in Section 5, with respect to the DKG $\mathcal{F}_{\text{KeyGen}}^\Delta$ ideal functionality given in Figure 1, an idealized PKI functionality $\mathcal{F}_{\text{PKI}}$

Participants	Comm. (unopt.)	Comm. (opt.)	Runtime (unopt.)	Runtime (opt.)
2	1.7 kb	1.7 kb	0.4 s	0.4 s
10	126 kb	22 kb	9.3 s	2.4 s
50	3.7 MB	223 kb	209 s	13.5 s
100	15.1 MB	699 kb	823 s	35.8 s

Table 5: Communication and estimated runtime for both the optimized and unoptimized variant of **Golden**. We consider running the DKG in an n -of- n configuration. Comm. represents the bandwidth overhead for each participant; kb represents kilobytes, and MB represents megabytes. Runtime represents the running time for each participant of performing both Round 0 and Round 1. Unopt. represents the unoptimized variant of **Golden** as presented in Figure 4, and Opt. represents the optimized variant discussed in Section 5.3.

Participants	Comm.	Runtime
2	59.4 kb	3.9 s
10	1.3 MB	5.6 s
50	27.8 MB	40.5 s
100	108.6 MB	130.8 s

Table 6: Benchmarks for the concrete efficiency of Groth’s DKG [48], using benchmarks and implementation from [53]. We consider running the DKG in an n -of- n configuration. Comm. represents the bandwidth overhead for each participant; kb represents kilobytes, and MB represents megabytes. Runtime represents the running time for each participant of performing both Round 0 and Round 1.

described in Appendix F, and the idealized eVRF functionality $\mathcal{F}_{\text{eVRF}}$ given in Appendix A. We show in the proof for Theorem 3 that the security of $\mathcal{F}_{\text{pki}}$ is equivalent to $\mathcal{F}_{\text{eVRF}}$ and the idealized zero-knowledge proof functionality $\mathcal{F}_{\text{zk}}^{\mathcal{R}_{EF}}$ (described in Appendix G) for the relation $\mathcal{R}_{\text{pki}} = \{(g, \text{PK}, (\text{sk})) : \text{PK} = g^{\text{sk}}\}$.

We first define **Golden** completely, with respect to $\mathcal{F}_{\text{pki}}$. We call this full, end-to-end protocol $\Pi_{\text{Golden-PKI}}$, which is simply **Golden**, conjoined with an initialize function, defined as follows:

Protocol 1 ($\Pi_{\text{Golden-PKI}}$):

- **Initialize:** Each party $i \in [n]$ interacts with the PKI functionality $\mathcal{F}_{\text{pki}}$ in Figure 8 with the following steps:
 1. Query **KeyGen** to $\mathcal{F}_{\text{pki}}$, receive $(\text{sk}_i^I, \text{PK}_i^I)$.
 2. Query **Register**(PK_i^I, i) to $\mathcal{F}_{\text{pki}}$.

3. Query Query to $\mathcal{F}_{\text{PKI}}$, receive $B = \{(j, \text{PK}_j^I)\}_{j \in [C]}$, such that $C \subseteq [n]$, from the functionality.
4. Store $\{(j, \text{PK}_j^I)\}_{j \in [C]}$.

Theorem 3. $\Pi_{\text{Golden-PKI}}$ securely realizes $\mathcal{F}_{\text{KeyGen}}^\Delta$ in the $(\mathcal{F}_{\text{zk}}^{\mathcal{R}_{\text{EF}}}, \mathcal{F}_{\text{eVRF}})$ -hybrid model, assuming $\text{eVRF}^\dagger$ as defined in Section 4 is a secure exponent verifiable random function, and $\mathcal{A}$ is a static adversary that corrupts up to $t - 1$ number of parties.

We give the full proof for Theorem 3 in Appendix H.

Proof Intuition. The core of the proof lies in defining a simulating algorithm SIM which interacts with the adversary $\mathcal{A}$ controlling $t - 1$ corrupt parties, and simulates honest parties in $\Pi_{\text{Golden-PKI}}$ with respect to a challenge $Y \in \mathbb{G}_{\text{out}}$ given by $\mathcal{F}_{\text{KeyGen}}^\Delta$. SIM follows the protocol honestly for all honest parties except for one simulated party $\tau \in \text{hon}$.

For party τ , SIM simulates Shamir secret sharing with respect to $g^{f_\tau(0)} = A_{\tau,0} = Y \cdot \prod_{k \in \text{hon}, k \neq \tau} (A_{k,0})^{-1}$, by picking the $t - 1$ corrupt parties' shares at random $\bar{x}_{\tau,j} \leftarrow \mathbb{Z}_p$ for $j \in \text{corr}$, and defining f_τ “in the exponent” as the degree- $t - 1$ polynomial defined by the t points $f_\tau(j) = \bar{x}_{\tau,j}$ and $A_{\tau,0}$. SIM then follows the protocol honestly to define $(R_{\tau,j}, z_{\tau,j})$ for $j \in \text{corr}$. However, SIM must also simulate $(R_{\tau,k}, z_{\tau,k})$ for all $k \in \text{hon}, k \neq \tau$ (even though SIM does not know $\bar{x}_{\tau,k}, k \in \text{hon}, k \neq \tau$). SIM does so by deriving the correct commitment to $g^{\bar{x}_{\tau,k}}$, by deriving $g^{\bar{x}_{\tau,k}} = \bar{X}_{\tau,k} = A_{\tau,0} \cdot \prod_{j=1}^{t-1} A_{\tau,j}^{k^j}$ via “interpolation in the exponent,” where $A_{\tau,j}$ is the VSS commitment to the j^{th} coefficient of the polynomial f_τ . SIM then samples $z_{\tau,k} \leftarrow \mathbb{Z}_p$ for all $k \in \text{hon}, k \neq \tau$, and sets $R_{\tau,k} = g^{z_{\tau,k}} \cdot \bar{X}_{\tau,k}$. The simulation is perfect due to the security of $\text{eVRF}^\dagger$, and because $z_{\tau,k}$ is sampled uniformly at random, so is likewise the product $g^{z_{\tau,k}} \cdot \bar{X}_{\tau,k}$.

After Round 0 is completed, because SIM knows corrupt parties' identity keys $\{\text{sk}_j^I\}_{j \in \text{corr}}$ from the initialize step for the PKI, SIM can decrypt and derive all corrupt parties' contributions $\{\omega_j\}_{j \in \text{corr}}$, such that $\Delta = \sum_{j \in \text{corr}} \omega_j$. Due to how $A_{\tau,0}$ is defined, honest parties' contributions to the DKG satisfy $\sum_{k \in \text{hon}} \omega_k = \text{dlog}(Y)$. Hence, $\text{PK} = Y \cdot \prod_{j \in \text{corr}} g^{\omega_j} = Y \cdot g^\Delta$, satisfying the requirements for $\mathcal{F}_{\text{KeyGen}}^\Delta$.

Acknowledgments

The authors would like to thank Faxing Wang for feedback. This work was supported by NSF grant 2239975 and generous gifts from Alpen Labs, Chaincode, and Google.

References

- [1] M. Abe and S. Fehr. “Adaptively Secure Feldman VSS and Applications to Universally-Composable Threshold Cryptography”. In: *CRYPTO 2004*. Ed. by M. K. Franklin. Vol. 3152. LNCS. Springer, 2004, pp. 317–334.

- [2] T. Attema, S. Fehr, and M. Klooß. “Fiat-shamir transformation of multi-round interactive proofs”. In: *Theory of Cryptography Conference*. 2022.
- [3] R. Bacho and A. Kavousi. “SoK: Dlog-Based Distributed Key Generation”. In: *IEEE Symposium on Security and Privacy, SP 2025, San Francisco, CA, USA, May 12-15, 2025*. Ed. by M. Blanton, W. Enck, and C. Nita-Rotaru. IEEE, 2025, pp. 614–632. DOI: 10.1109/SP61157.2025.00127. URL: <https://doi.org/10.1109/SP61157.2025.00127>.
- [4] A. Bassa. *Soundness Proof for Eagen’s Proof of Sums of Points*. https://repo.getmonero.org/-/project/54/uploads/eb1bf5b4d4855a3480c38abf895bd8e8/Veridise_Divisor_Proofs.pdf. 2022.
- [5] M. Bellare, E. C. Crites, C. Komlo, M. Maller, S. Tessaro, and C. Zhu. “Better than Advertised Security for Non-interactive Threshold Signatures”. In: *CRYPTO 2022*. Ed. by Y. Dodis and T. Shrimpton. Vol. 13510. LNCS. Springer, 2022, pp. 517–550.
- [6] M. Bellare and G. Neven. “Multi-signatures in the plain public-key model and a general forking lemma”. In: *Proceedings of the 13th ACM conference on Computer and communications security*. 2006, pp. 390–399.
- [7] D. J. Bernstein. “Curve25519: New Diffie–Hellman Speed Records”. In: *Public Key Cryptography - PKC 2006*. Ed. by M. Yung, Y. Dodis, A. Kiayias, and T. Malkin. Vol. 3958. LNCS. Springer, 2006, pp. 207–228.
- [8] N. Bitansky, R. Canetti, A. Chiesa, and E. Tromer. “From extractable collision resistance to succinct non-interactive arguments of knowledge, and back again”. In: *Innovations in Theoretical Computer Science 2012, Cambridge, MA, USA, January 8-10, 2012*. Ed. by S. Goldwasser. ACM, 2012, pp. 326–349. DOI: 10.1145/2090236.2090263. URL: <https://doi.org/10.1145/2090236.2090263>.
- [9] A. Boldyreva. “Threshold Signatures, Multisignatures and Blind Signatures Based on the Gap-Diffie-Hellman-Group Signature Scheme”. In: *Public Key Cryptography - PKC 2003, 6th International Workshop on Theory and Practice in Public Key Cryptography, Miami, FL, USA, January 6-8, 2003, Proceedings*. Ed. by Y. Desmedt. Vol. 2567. Lecture Notes in Computer Science. Springer, 2003, pp. 31–46. DOI: 10.1007/3-540-36288-6_3. URL: https://doi.org/10.1007/3-540-36288-6_3.
- [10] D. Boneh, T. Datta, R. Fernando, K. Nazirkhanova, and A. Tomescu. “Dekart-Proof: Efficient Vector Range Proofs and Their Applications”. In: *Cryptology ePrint Archive* (2025).
- [11] D. Boneh, M. Drijvers, and G. Neven. “Compact multi-signatures for smaller blockchains”. In: *International Conference on the Theory and Application of Cryptology and Information Security*. Springer. 2018, pp. 435–464.
- [12] D. Boneh, I. Haitner, Y. Lindell, and G. Segev. “Exponent-VRFs and Their Applications”. In: *EUROCRYPT 2025 - 44th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Madrid, Spain, May 4-8, 2025, Proceedings, Part VII*. Ed. by S. Fehr and P. Fouque. Vol. 15607. Lecture Notes in Computer Science. Springer, 2025, pp. 195–224. DOI: 10.1007/978-3-031-91098-2_8. URL: https://doi.org/10.1007/978-3-031-91098-2_8.
- [13] L. Brandão and R. Peralta. *NIST First Call for Multi-Party Threshold Schemes*. <https://nvlpubs.nist.gov/nistpubs/ir/2023/NIST.IR.8214C.ipd.pdf>. 2023.
- [14] B. Bünz. *Improving the Privacy Scalability and Ecological Impact of Blockchains*. Stanford University, 2023.

- [15] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell. “Bulletproofs: Short proofs for confidential transactions and more”. In: *2018 IEEE symposium on security and privacy (SP)*. IEEE. 2018, pp. 315–334.
- [16] R. Canetti, R. Gennaro, S. Jarecki, H. Krawczyk, and T. Rabin. “Adaptive Security for Threshold Cryptosystems”. In: *CRYPTO ’99, 19th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 1999*. Ed. by M. J. Wiener. Vol. 1666. LNCS. Springer, 1999, pp. 98–115.
- [17] R. Canetti and T. Rabin. “Universal Composition with Joint State”. In: *CRYPTO 2003, 23rd Annual International Cryptology Conference, Santa Barbara, California, USA, August 17-21, 2003, Proceedings*. Ed. by D. Boneh. Vol. 2729. Lecture Notes in Computer Science. Springer, 2003, pp. 265–281. DOI: 10.1007/978-3-540-45146-4_16. URL: https://doi.org/10.1007/978-3-540-45146-4_16.
- [18] R. Canetti, D. Shahaf, and M. Vald. “Universally Composable Authentication and Key-Exchange with Global PKI”. In: *Public-Key Cryptography - PKC 2016 - 19th IACR International Conference on Practice and Theory in Public-Key Cryptography, Taipei, Taiwan, March 6-9, 2016, Proceedings, Part II*. Ed. by C. Cheng, K. Chung, G. Persiano, and B. Yang. Vol. 9615. Lecture Notes in Computer Science. Springer, 2016, pp. 265–296. DOI: 10.1007/978-3-662-49387-8_11. URL: https://doi.org/10.1007/978-3-662-49387-8_11.
- [19] I. Cascudo and B. David. “Publicly Verifiable Secret Sharing Over Class Groups and Applications to DKG and YOSO”. In: *Advances in Cryptology - EUROCRYPT 2024 - 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zurich, Switzerland, May 26-30, 2024, Proceedings, Part V*. Ed. by M. Joye and G. Leander. Vol. 14655. Lecture Notes in Computer Science. Springer, 2024, pp. 216–248. DOI: 10.1007/978-3-031-58740-5_8. URL: https://doi.org/10.1007/978-3-031-58740-5_8.
- [20] I. Cascudo and B. David. “SCRAPE: Scalable Randomness Attested by Public Entities”. In: *Applied Cryptography and Network Security (ACNS) 2017*. Ed. by D. Gollmann, A. Miyaji, and H. Kikuchi. Vol. 10355. LNCS. Springer, 2017, pp. 537–556.
- [21] I. Cascudo, B. David, O. Shlomovits, and D. Varlakov. “Mt. Random: Multi-tiered Randomness Beacons”. In: *Applied Cryptography and Network Security - 21st International Conference, ACNS 2023, Kyoto, Japan, June 19-22, 2023*. Ed. by M. Tibouchi and X. Wang. Vol. 13906. LNCS. Springer, 2023, pp. 645–674. DOI: 10.1007/978-3-031-33491-7_24. URL: https://doi.org/10.1007/978-3-031-33491-7_24.
- [22] D. Cash, E. Kiltz, and V. Shoup. “The Twin Diffie–Hellman Problem and Applications”. In: *EUROCRYPT 2008*. Ed. by N. P. Smart. Vol. 4965. LNCS. Springer, 2008, pp. 127–145.
- [23] D. Chaum and T. P. Pedersen. “Wallet databases with observers”. In: *Annual international cryptology conference*. 1992.
- [24] M. Chen, P. Dey, C. Ganesh, P. Mukherjee, P. Sarkar, and S. Sasmal. “Universally Composable Non-interactive Zero-Knowledge from Sigma Protocols via a New Straight-Line Compiler”. In: *Public-Key Cryptography - PKC 2025 - 28th IACR International Conference on Practice and Theory of Public-Key Cryptography, Røros, Norway, May 12-15, 2025, Proceedings, Part I*. Ed. by T. Jager and J. Pan. Vol. 15674. LNCS. Springer, 2025, pp. 381–417. DOI: 10.1007/978-3-031-91820-9_13. URL: https://doi.org/10.1007/978-3-031-91820-9_13.
- [25] K. Choi, A. Manoj, and J. Bonneau. “Sok: Distributed randomness beacons”. In: *2023 IEEE Symposium on Security and Privacy (SP)*. 2023.

- [26] H. Chu, P. Gerhart, T. Ruffing, and D. Schröder. “Practical Schnorr Threshold Signatures Without the Algebraic Group Model”. In: *CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023*. Ed. by H. Handschuh and A. Lysyanskaya. Vol. 14081. Lecture Notes in Computer Science. Springer, 2023, pp. 743–773. DOI: 10.1007/978-3-031-38557-5_24. URL: https://doi.org/10.1007/978-3-031-38557-5_24.
- [27] H. Chung, K. Han, C. Ju, M. Kim, and J. H. Seo. “Bulletproofs+: Shorter proofs for a privacy-enhanced distributed ledger”. In: *Ieee Access* (2022).
- [28] arkworks contributors. *arkworks zkSNARK ecosystem*. 2022. URL: <https://arkworks.rs>.
- [29] R. Cramer, I. Damgård, and Y. Ishai. “Share Conversion, Pseudorandom Secret-Sharing and Applications to Secure Computation”. In: *Theory of Cryptography, Second Theory of Cryptography Conference, TCC 2005, Cambridge, MA, USA, February 10-12, 2005, Proceedings*. Ed. by J. Kilian. Vol. 3378. Lecture Notes in Computer Science. Springer, 2005, pp. 342–362. DOI: 10.1007/978-3-540-30576-7_19. URL: https://doi.org/10.1007/978-3-540-30576-7_19.
- [30] E. Crites, C. Komlo, and M. Maller. *How to Prove Schnorr Assuming Schnorr: Security of Multi- and Threshold Signatures*. Cryptology ePrint Archive, Paper 2021/1375. 2021. URL: <https://eprint.iacr.org/2021/1375>.
- [31] E. Crites, C. Komlo, and M. Maller. *On the Adaptive Security of Key-Unique Threshold Signatures*. Cryptology ePrint Archive, Paper 2025/943. 2025. URL: <https://eprint.iacr.org/2025/943>.
- [32] S. Das, L. Ren, and Z. Yang. “Adaptively Secure Threshold ElGamal Decryption from DDH”. In: *IACR Cryptol. ePrint Arch.* (2025).
- [33] Y. Desmedt. “Threshold cryptosystems”. In: *international workshop on the theory and application of cryptographic techniques*. 1992.
- [34] DFINITY. *Applied Crypto: Introducing Noninteractive Distributed Key Generation*. <https://medium.com/dfinity/applied-crypto-one-public-key-for-the-internet-computer-ni-dkg-4af800db869d>. 2025.
- [35] Y. Dodis and A. Yampolskiy. “A verifiable random function with short proofs and keys”. In: *International Workshop on Public Key Cryptography*. 2005.
- [36] L. Eagen. “Zero knowledge proofs of elliptic curve inner products from principal divisors and weil reciprocity”. In: *Cryptology ePrint Archive* (2022).
- [37] J. Ernstberger et al. “zk-Bench: A Toolset for Comparative Evaluation and Performance Benchmarking of SNARKs”. In: *Security and Cryptography for Networks*. Ed. by C. Galdi and D. H. Phan. Springer Nature Switzerland, 2024, pp. 46–72. ISBN: 978-3-031-71070-4.
- [38] P. Feldman. “A Practical Scheme for Non-interactive Verifiable Secret Sharing”. In: *28th Annual Symposium on Foundations of Computer Science*. 1987.
- [39] H. Feng, T. Mai, and Q. Tang. “Scalable and Adaptively Secure Any-Trust Distributed Key Generation and All-hands Checkpointing”. In: *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS 2024, Salt Lake City, UT, USA, October 14-18, 2024*. Ed. by B. Luo, X. Liao, J. Xu, E. Kirda, and D. Lie. ACM, 2024, pp. 2636–2650. DOI: 10.1145/3658644.3690253. URL: <https://doi.org/10.1145/3658644.3690253>.
- [40] A. Fiat and A. Shamir. “How to Prove Yourself: Practical Solutions to Identification and Signature Problems”. In: *CRYPTO ’86*. Vol. 263. LNCS. Springer, 1986, pp. 186–194.

- [41] M. Fischlin. “Communication-Efficient Non-interactive Proofs of Knowledge with Online Extractors”. In: *CRYPTO 2005*. Ed. by V. Shoup. Vol. 3621. LNCS. Springer, 2005, pp. 152–168.
- [42] P. Fouque and J. Stern. “One Round Threshold Discrete-Log Key Generation without Private Channels”. In: *Public Key Cryptography, 4th International Workshop on Practice and Theory in Public Key Cryptography, PKC 2001, Cheju Island, Korea, February 13-15, 2001, Proceedings*. Ed. by K. Kim. Vol. 1992. Lecture Notes in Computer Science. Springer, 2001, pp. 300–316. DOI: 10.1007/3-540-44586-2_22. URL: https://doi.org/10.1007/3-540-44586-2_22.
- [43] E. S. V. Freire, D. Hofheinz, E. Kiltz, and K. G. Paterson. “Non-Interactive Key Exchange”. In: *Public-Key Cryptography - PKC 2013*. Ed. by K. Kurosawa and G. Hanaoka. Vol. 7778. LNCS. Springer, 2013, pp. 254–271.
- [44] S. D. Galbraith, P. Wang, and F. Zhang. *Computing Elliptic Curve Discrete Logarithms with Improved Baby-step Giant-step Algorithm*. Cryptology ePrint Archive, Paper 2015/605. 2015. URL: <https://eprint.iacr.org/2015/605>.
- [45] R. Gao, Z. Wan, Y. Hu, and H. Wang. “A succinct range proof for polynomial-based vector commitment”. In: *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 2024.
- [46] R. Gennaro, S. Jarecki, H. Krawczyk, and T. Rabin. “Secure Applications of Pedersen’s Distributed Key Generation Protocol”. In: *CT-RSA 2003*. 2003, pp. 373–390. ISBN: 978-3-540-36563-1.
- [47] R. Gennaro, S. Jarecki, H. Krawczyk, and T. Rabin. “Secure Distributed Key Generation for Discrete-Log Based Cryptosystems”. In: *J. Cryptol.* 20.1 (2007), pp. 51–83.
- [48] J. Groth. *Non-interactive distributed key generation and key resharing*. Cryptology ePrint Archive, Paper 2021/339. 2021. URL: <https://eprint.iacr.org/2021/339>.
- [49] K. Gurkan, P. Jovanovic, M. Maller, S. Meiklejohn, G. Stern, and A. Tomescu. “Aggregatable Distributed Key Generation”. In: *EUROCRYPT 2021*. Ed. by A. Canteaut and F. Standaert. Vol. 12696. LNCS. Springer, 2021, pp. 147–176.
- [50] R. Impagliazzo, L. A. Levin, and M. Luby. “Pseudo-random generation from one-way functions”. In: *Proceedings of the twenty-first annual ACM symposium on Theory of computing*. 1989, pp. 12–24.
- [51] S. Jarecki, H. Krawczyk, and J. Resch. “Threshold Partially-Oblivious PRFs with Applications to Key Management”. In: (2018). URL: <https://eprint.iacr.org/2018/733>.
- [52] S. Jarecki and A. Lysyanskaya. “Adaptively Secure Threshold Cryptography: Introducing Concurrency, Removing Erasures”. In: *Advances in Cryptology - EUROCRYPT 2000, International Conference on the Theory and Application of Cryptographic Techniques, Bruges, Belgium, May 14-18, 2000, Proceeding*. Ed. by B. Preneel. Vol. 1807. Lecture Notes in Computer Science. Springer, 2000, pp. 221–242. DOI: 10.1007/3-540-45539-6_16. URL: https://doi.org/10.1007/3-540-45539-6_16.
- [53] A. Kate, E. V. Mangipudi, P. Mukherjee, H. Saleem, and S. A. K. Thyagarajan. “Non-interactive VSS using Class Groups and Application to DKG”. In: *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 2024, pp. 4286–4300.
- [54] J. Katz. “Round Optimal Robust Distributed Key Generation”. In: *IACR Cryptol. ePrint Arch.* (2023), p. 1094. URL: <https://eprint.iacr.org/2023/1094>.

- [55] A. Kavousi, Z. Wang, and P. Jovanovic. “SoK: public randomness”. In: *2024 IEEE 9th European Symposium on Security and Privacy (EuroS&P)*. 2024.
- [56] C. Komlo and I. Goldberg. “FROST: Flexible Round-Optimized Schnorr Threshold Signatures”. In: *Selected Areas in Cryptography - SAC 2020*. Ed. by O. Dunkelman, M. J. J. Jr., and C. O’Flynn. Vol. 12804. LNCS. Springer, 2020, pp. 34–65.
- [57] E. Kushilevitz, Y. Lindell, and T. Rabin. “Information-Theoretically Secure Protocols and Security under Composition”. In: *SIAM J. Comput.* 39.5 (2010), pp. 2090–2112.
- [58] League of Entropy. *Distributed randomness beacon*. <https://drand.love/>. 2019.
- [59] Y. Lindell. *Simple Three-Round Multiparty Schnorr Signing with Full Simulatability*. Cryptology ePrint Archive, Report 2022/374. <https://ia.cr/2022/374>. 2022.
- [60] A. Miller, Y. Xia, K. Croman, E. Shi, and D. Song. “The Honey Badger of BFT Protocols”. In: *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, Vienna, Austria, October 24–28, 2016*. Ed. by E. R. Weippl, S. Katzenbeisser, C. Kruegel, A. C. Myers, and S. Halevi. ACM, 2016, pp. 31–42. DOI: 10.1145/2976749.2978399. URL: <https://doi.org/10.1145/2976749.2978399>.
- [61] M. Naor, B. Pinkas, and O. Reingold. “Distributed Pseudo-random Functions and KDCs”. In: *EUROCRYPT ’99*. Ed. by J. Stern. Vol. 1592. LNCS. Springer, 1999, pp. 327–346.
- [62] J. Nick, T. Ruffing, Y. Seurin, and P. Wuille. “MuSig-DN: Schnorr Multi-Signatures with Verifiably Deterministic Nonces”. In: *CCS ’20: 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, USA, November 9–13, 2020*. Ed. by J. Ligatti, X. Ou, J. Katz, and G. Vigna. ACM, 2020, pp. 1717–1731. DOI: 10.1145/3372297.3417236. URL: <https://doi.org/10.1145/3372297.3417236>.
- [63] V. Nikolaenko, S. Ragsdale, J. Bonneau, and D. Boneh. “Powers-of-Tau to the People: Decentralizing Setup Ceremonies”. In: *Applied Cryptography and Network Security - 22nd International Conference, ACNS 2024, Abu Dhabi, United Arab Emirates, March 5–8, 2024, Proceedings, Part III*. Ed. by C. Pöpper and L. Batina. Vol. 14585. Lecture Notes in Computer Science. Springer, 2024, pp. 105–134. DOI: 10.1007/978-3-031-54776-8_5. URL: https://doi.org/10.1007/978-3-031-54776-8_5.
- [64] P. Paillier. “Public-Key Cryptosystems Based on Composite Degree Residuosity Classes”. In: *Advances in Cryptology - EUROCRYPT ’99, International Conference on the Theory and Application of Cryptographic Techniques, Prague, Czech Republic, May 2–6, 1999, Proceeding*. Ed. by J. Stern. Vol. 1592. Lecture Notes in Computer Science. Springer, 1999, pp. 223–238. DOI: 10.1007/3-540-48910-X_16. URL: https://doi.org/10.1007/3-540-48910-X_16.
- [65] L. Parker. *R1CS gadget for the 2^k -bit scaling of a fixed generator in seven multiplicative constraints*. <https://github.com/kayabaNerve/fcmp-plus-plus-paper/blob/divisor-paper/divisors.pdf>. 2024.
- [66] T. P. Pedersen. “A Threshold Cryptosystem without a Trusted Party (Extended Abstract)”. In: *EUROCRYPT 1991*. Ed. by D. W. Davies. Vol. 547. LNCS. Springer, 1991, pp. 522–526.
- [67] N. Pippenger. “On the evaluation of powers and monomials”. In: *SIAM Journal on Computing* (1980).

- [68] T. Ruffing and J. Nick. *ChillDKG: Distributed Key Generation for FROST*. 2025. URL: <https://github.com/BlockstreamResearch/bip-frost-dkg>.
- [69] B. Schoenmakers. “A Simple Publicly Verifiable Secret Sharing Scheme and Its Application to Electronic”. In: *CRYPTO ’99, 19th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 1999*. Ed. by M. J. Wiener. Vol. 1666. Lecture Notes in Computer Science. Springer, 1999, pp. 148–164. DOI: 10.1007/3-540-48405-1_10. URL: https://doi.org/10.1007/3-540-48405-1_10.
- [70] G. Segev. “Bulletproofs for R1CS: Bridging the Completeness-Soundness Gap and a ZK Extension”. In: *Cryptology ePrint Archive* (2025).
- [71] A. Shamir. “How to Share a Secret”. In: *Commun. ACM* 22.11 (1979), pp. 612–613.
- [72] E. Syta et al. “Scalable Bias-Resistant Distributed Randomness”. In: *2017 IEEE Symposium on Security and Privacy, SP 2017, San Jose, CA, USA, May 22-26, 2017*. IEEE Computer Society, 2017, pp. 444–460. DOI: 10.1109/SP.2017.45. URL: <https://doi.org/10.1109/SP.2017.45>.
- [73] N. Trapp and D. Ongaro. *Juicebox Protocol: Distributed Storage and Recovery of Secrets Using Simple PIN Authentication*. Cryptology ePrint Archive, Paper 2025/348. 2025. URL: <https://eprint.iacr.org/2025/348>.
- [74] F. Wang, S. Cohnen, and J. Bonneau. “SoK: Trusted setups for powers-of-tau strings”. In: *International Conference on Financial Cryptography and Data Security*. 2025.

A Simulation-Based Security for eVRFs

We review the simulation-based notion of security for an eVRF by Boneh et al. [12], as the idealized functionality $\mathcal{F}_{\text{eVRF}}$. The authors prove this functionality is in fact implied by the eVRF game-based notions in conjunction with a zero-knowledge proof of knowledge of the private key. We show $\mathcal{F}_{\text{eVRF}}$ in Figure 5.

As observed by Boneh et al. [12], the environment can distinguish between different eVRF instances by accepting as input the verification key PK as a parameter. Hence, the functionality $\mathcal{F}_{\text{eVRF}}$ is presented without requiring an additional session identifier sid.

B Non-Interactive Key Exchange (NIKE)

Our new eVRF builds upon a (two-party) Non-Interactive Key Exchange (NIKE) [7, 22, 43] scheme, as defined below. We build upon prior NIKE definitions in the literature [22, 43]; these notions allow an adversary to act as an active participant, and require it to perform a distinguishing attack on a shared key between the public keys associated with two identities of the adversary’s choosing.

Definition 3. A Non-Interactive Key Exchange (NIKE) NIKE is the tuple $\text{NIKE} = (\text{Setup}, \text{KeyGen}, \text{GetSharedKey})$, where

- $\text{Setup}(1^\lambda) \rightarrow \text{par}$: Accepts as input the security parameter λ , and outputs public parameters par , which are implicitly given as input to all other algorithms.

$$\mathcal{F}_{\text{eVRF}}$$

- **Init:** On input δ , where δ is some constant indicating a timeout, initialize the functionality, by performing the following steps:
 1. Define $Q_m \leftarrow \emptyset, Q_K^{(1)} \leftarrow \emptyset, Q_K^{(2)} \leftarrow \emptyset$.
- **Register:** On query input $(j, \text{sid}_j, *)$:
 1. Output $\perp$ if id' is not the identifier for party id .
 2. If $Q_K^{(1)}[\text{sid}_j] \neq \perp$, return $\perp$.
 3. If $j \in \text{corr}$ and $Q_K^{(2)}[\text{sid}_j] \neq \perp$, return $\perp$.
 4. For all parties, set $Q_K^{(1)}[\text{sid}_j] = j$.
 5. If $j \in \text{corr}$ and the input is (j, sid_j, f_j) , set $Q_K^{(2)}[\text{sid}_j] = (j, f_j)$.
- **Query:** On a query (with no input), wait δ amount of time (where δ is the constant defined above), to give all parties enough time to complete registration. Then, set $B \leftarrow Q_K^{(1)}$, such that $B = \{(j, \text{sid}_j)\}_{j \in C}$, such that $C \subseteq [n]$. Output B .
- **Evaluate:** On query with input $(i, \text{sid}, \text{msg})$:
 1. If $Q_K^{(1)}[\text{sid}] \neq i$, then return $\perp$.
 2. If $i \in \text{hon}$, then do the following:
 - (a) If $(i, \text{msg}) \in Q_m$, parse $(\alpha, A) \leftarrow Q_m[i, \text{msg}]$.
 - (b) Else, sample $\alpha \leftarrow_s \mathbb{Z}_p$, and derive $A \leftarrow g^\alpha$. Store $Q_m[i, \text{msg}] \leftarrow (\alpha, A)$.
 - (c) Return (α, A) to party i and send (msg, A) to all parties.
 3. Else, $i \in \text{corr}$, so do the following:
 - (a) Get $(i, f_i) \leftarrow Q_K^{(2)}[\text{sid}]$.
 - (b) Derive $\alpha \leftarrow_s f_i(\text{msg})$ and $A \leftarrow g^\alpha$.
 - (c) Send (msg, A) to all parties.

Fig. 5: Ideal functionality for an exponent VRF, as defined by Boneh et al. [12]. Note we define **Query** as a separate oracle to learn the registered identities for all parties, instead of broadcasting directly after registration as done by Boneh et al. [12].

- **KeyGen()** $\rightarrow (\text{sk}, \text{PK})$: Generates a secret key sk and corresponding public key PK .
- **GetSharedKey**(sk_1, PK_2) $\rightarrow S$: Accepts the first party's secret key sk_1 and the second party's public key PK_2 . Outputs the shared key S generated by the combination of one party's secret key sk_1 and the other party's public key PK_2 .

- For correctness, we require that, for all $(\text{sk}_1, \text{PK}_1) \leftarrow^* \text{NIKE.KeyGen}(\lambda)$ and $(\text{sk}_2, \text{PK}_2) \leftarrow^* \text{NIKE.KeyGen}(\lambda)$, the following conditions hold:

$$\text{NIKE.GetSharedKey}(\text{sk}_1, \text{PK}_2) = \text{NIKE.GetSharedKey}(\text{sk}_2, \text{PK}_1)$$

In order to be secure, a NIKE must be *session-key unrecoverable*, which we discuss next.

Session-Key Unrecoverability. A NIKE which is *session-key unrecoverable* ensures that given the public keys of two parties, it is hard for an adversary to distinguish the shared secret from a secret that was sampled uniformly at random.

The advantage of an adversary $\mathcal{A}$ against NIKE in the unrecoverability experiment as defined in Figure 6 is

$$\text{Adv}_{\text{NIKE}, \mathcal{A}}^{\text{unrec}}(\lambda) = |\Pr[\text{Exp}_{\text{NIKE}, 0, \mathcal{A}}^{\text{unrec}}(\lambda) = 1] - \Pr[\text{Exp}_{\text{NIKE}, 1, \mathcal{A}}^{\text{unrec}}(\lambda) = 1]|$$

Definition 4. A NIKE is **session-key unrecoverable** if for all probabilistic polynomial time adversaries $\mathcal{A}$, the function $\text{Adv}_{\text{NIKE}, \mathcal{A}}^{\text{unrec}}(\lambda)$ is negligible.

C Leftover Hash Lemma

We need a simple extractor within our eVRF construction to ensure that our eVRF satisfies pseudorandomness. To show this in the security proof, we use the leftover hash lemma [50], which bounds the statistical distance between corresponding probability distributions.

Statistical Distance. Let X and Y be two discrete random variables over a countable set A . The statistical distance between X and Y is the quantity

$$\Delta(X, Y) = \frac{1}{2} \sum_{a \in A} |\Pr\{X = a\} - \Pr\{Y = a\}|.$$

For any predicate $p : A \rightarrow \{0, 1\}$, the following inequality holds:

$$|\Pr[p(X) = 1] - \Pr[p(Y) = 1]| \leq \Delta(X, Y).$$

Lemma 1 (Leftover Hash Lemma [50]). Let $\text{ext} : \mathcal{K} \times \mathcal{X} \rightarrow \mathbb{F}_q$ be a universal hash, i.e. $\Pr_k[\text{ext}(k, x) = \text{ext}(k, x')] \leq \frac{1}{q}$ for all distinct $x, x' \in \mathcal{X}$. Let $K, X_1, \dots, X_m$ be mutually independent random variables, where K is uniformly distributed over $\mathcal{K}$, and each X_i is distributed over $\mathcal{X}$ such that for all i , the guessing probability $\max_{x \in \mathcal{X}} \Pr[X_i = x] \leq \gamma$. Let Δ be the statistical distance between

$$(K, \text{ext}(K, X_1), \dots, \text{ext}(K, X_m))$$

and the uniform distribution on $\mathcal{K} \times \mathcal{X}^m$. Then

$$\Delta \leq \frac{m}{2} \cdot \sqrt{\gamma q}$$

$\text{Exp}_{\text{NIKE}, b, \mathcal{A}}^{\text{unrec}}(\lambda)$	$\mathcal{O}^{\text{Reveal}}(\text{id}_A, \text{id}_B)$
1 : par $\leftarrow \text{NIKE.Setup}(1^\lambda)$ 2 : $Q_{\text{hon}} \leftarrow \emptyset; Q_{\text{corr}} \leftarrow \emptyset$ 3 : $b' \leftarrow \$ \mathcal{A}^{\mathcal{O}^{\text{RegHon, RegCorr, Extract, Reveal, Test}}}(\text{par})$ 4 : return b'	1 : <i>// Requires id_A to be honest wlog</i> 2 : if $Q_{\text{hon}}[\text{id}_A] = \perp$ 3 : return $\perp$ 4 : $(\text{sk}_{\text{id}_A}, \text{PK}_{\text{id}_A}) \leftarrow Q_{\text{hon}}[\text{id}_A]$ 5 : if $Q_{\text{hon}}[\text{id}_B] \neq \perp$ 6 : $\text{PK}_{\text{id}_B} \leftarrow Q_{\text{hon}}[\text{id}_B]$ 7 : else 8 : $\text{PK}_{\text{id}_B} \leftarrow Q_{\text{corr}}[\text{id}_B]$ 9 : $S \leftarrow \text{NIKE.GetSharedKey}(\text{sk}_{\text{id}_A}, \text{PK}_{\text{id}_B})$ 10 : return S
$\mathcal{O}^{\text{RegHon}}(\text{id})$	$\mathcal{O}^{\text{Test}}(\text{id}_A, \text{id}_B)$
1 : if $Q_{\text{hon}}[\text{id}] \neq \perp$ 2 : $(\text{sk}_{\text{id}}, \text{PK}_{\text{id}}) \leftarrow Q_{\text{hon}}[\text{id}]$ 3 : else 4 : $(\text{sk}_{\text{id}}, \text{PK}_{\text{id}}) \leftarrow \$ \text{NIKE.KeyGen}(\lambda)$ 5 : $Q_{\text{hon}}[\text{id}] \leftarrow (\text{sk}_{\text{id}}, \text{PK}_{\text{id}})$ 6 : return PK_{id}	1 : if $\text{id}_A = \text{id}_B$ 2 : return $\perp$ 3 : if $Q_{\text{hon}}[\text{id}_A] = \perp$ or $Q_{\text{hon}}[\text{id}_B] = \perp$ 4 : return $\perp$ 5 : $(\text{sk}_{\text{id}_A}, \text{PK}_{\text{id}_A}) \leftarrow Q_{\text{hon}}[\text{id}_A]$ 6 : $(\text{sk}_{\text{id}_B}, \text{PK}_{\text{id}_B}) \leftarrow Q_{\text{hon}}[\text{id}_B]$ 7 : if $b = 0$ 8 : $S \leftarrow \text{NIKE.GetSharedKey}(\text{sk}_{\text{id}_A}, \text{PK}_{\text{id}_B})$ 9 : else <i>// $b = 1$ case</i> 10 : $S \leftarrow \$ \hat{\mathcal{S}}$ 11 : <i>// $\hat{\mathcal{S}}$ is the domain of shared keys</i> 12 : return S
$\mathcal{O}^{\text{RegCorr}}(\text{id}, \text{PK}_{\text{id}})$	
1 : $Q_{\text{corr}}[\text{id}] \leftarrow \text{PK}_{\text{id}}$	
$\mathcal{O}^{\text{Extract}}(\text{id})$	
1 : return $\perp$ if $Q_{\text{hon}}[\text{id}] = \perp$ 2 : $(\text{sk}_{\text{id}}, \text{PK}_{\text{id}}) \leftarrow Q_{\text{hon}}[\text{id}]$ 3 : $Q_{\text{hon}}[\text{id}] \leftarrow \perp; Q_{\text{corr}}[\text{id}] \leftarrow \text{PK}_{\text{id}}$ 4 : return $(\text{sk}_{\text{id}}, \text{PK}_{\text{id}})$	

Fig. 6: Unrecoverability experiment defining the advantage of an adversary $\mathcal{A}$ against a non-interactive key exchange (NIKE) NIKE. To prevent trivial wins, the adversary cannot make queries to $\mathcal{O}^{\text{Reveal}}$ with inputs to $\mathcal{O}^{\text{Test}}$, and similarly, cannot make $\mathcal{O}^{\text{Extract}}$ queries for identities involved in $\mathcal{O}^{\text{Test}}$ queries.

D Proof of Theorem 2

Proof. While *correctness* holds by construction, *verifiability* and *simulatability* follow from the soundness and the zero-knowledge property of the argument system for $\text{eVRF}^\dagger$, respectively.

As for *pseudorandomness*, we show an explicit bound while analyzing our construction via the following game hops:

Game 0. This is the eVRF pseudorandomness experiment (Figure 2) with $b = 0$, instantiated with $\text{eVRF}^\dagger$. Let $\mathcal{A}$ be an adversary playing against Game 0, and let W_0 be the event that $\mathcal{A}$ wins Game 0 (and similarly W_i for Game i). Then the probability that $\mathcal{A}$ wins Game 0 is denoted by $\Pr[W_0]$.

Game 1. Instead of $S \leftarrow \text{PK}_2^{\text{sk}_1}$, we take S to be a truly random group element.

Reduction to Session-Key Unrecoverability of the NIKE. We now define a reduction $\mathcal{B}_1$ that uses $\mathcal{A}$ as a subroutine to win against the session-key unrecoverability game of the Diffie-Hellman NIKE, as shown in Figure 6.

To begin, $\mathcal{B}_1$ accepts the challenge instance $(\text{PK}_1, \text{PK}_2, \tilde{S})$ from the Diffie-Hellman NIKE session-key unrecoverability challenger. $\mathcal{B}_1$ then computes the following:

- $\tilde{k} \leftarrow \tilde{S} \cdot X \in \mathbb{F}_p$
- $\tilde{T}_1 \leftarrow H_{\mathbb{G}_{\text{in}},1}(\text{msg})^{\tilde{k}} \in \mathbb{G}_{\text{in}}$
- $\tilde{T}_2 \leftarrow H_{\mathbb{G}_{\text{in}},2}(\text{msg})^{\tilde{k}} \in \mathbb{G}_{\text{in}}$
- $\tilde{r}_1 \leftarrow \tilde{T}_1 \cdot X \in \mathbb{F}_p$
- $\tilde{r}_2 \leftarrow \tilde{T}_2 \cdot X \in \mathbb{F}_p$
- $\tilde{r} \leftarrow \beta \cdot \tilde{r}_1 + \tilde{r}_2 \in \mathbb{F}_p$

where msg and β are randomly sampled by $\mathcal{B}_1$.

$\mathcal{B}_1$ can act as a challenger to $\mathcal{A}$ with the above values and output whichever result that $\mathcal{A}$ outputs, in which case the simulation is perfect; $\mathcal{A}$'s Game 1 corresponds to the $b = 1$ case (for $\mathcal{B}_1$'s challenge instance) while Game 0 corresponds to the $b = 0$ case.

Difference between Game 0 and Game 1. As a result, we bound the distinguishing advantage between Game 0 and Game 1 by:

$$|\Pr[W_1] - \Pr[W_0]| \leq |\Pr[\text{Exp}_{\text{DH},1,\mathcal{A}}^{\text{unrec}}(\lambda) = 1] - \Pr[\text{Exp}_{\text{DH},0,\mathcal{A}}^{\text{unrec}}(\lambda) = 1]| \quad (1)$$

$$= \text{Adv}_{\text{DH},\mathcal{B}_1}^{\text{unrec}}(\lambda) \quad (2)$$

Game 2. Instead of $r \leftarrow \beta \cdot r_1 + r_2 \in \mathbb{F}_p$, we sample a truly random $r \in \mathbb{F}_p$ in Game 2. This is essentially the eVRF pseudorandomness experiment (Figure 2) with $b = 1$. Unlike a classical DDH PRF, our $H_{\mathbb{G}_{\text{in}},1}(\text{msg})^k$ (and similarly $H_{\mathbb{G}_{\text{in}},2}(\text{msg})^k$) from Game 1 has a more restricted domain for k . Specifically, $k \leftarrow S \cdot X \in \mathbb{F}_p$ for some random $S \in \mathbb{G}_{\text{in}}$ means that the domain of k (denoted by D_k) consists of $\frac{s-1}{2}$ values in $\mathbb{F}_p$ and k is uniformly distributed on D_k .

Difference between Game 1 and Game 2. Considering this, we are able to bound the distinguishing advantage between Game 1 and Game 2 using the leftover hash lemma (Appendix C) directly as follows. To apply the leftover hash lemma to $(\beta, r = \beta \cdot r_1 + r_2)$, we need to bound the guessing probability of (r_1, r_2) the values of which are derived from applying the random oracles $H_{\mathbb{G}_{\text{in}},1}$ and $H_{\mathbb{G}_{\text{in}},2}$ on

msg , respectively. For r_1 (and similarly for r_2), we bound the guessing probability of r_1 by

$$\max_{z \in \mathbb{F}_p} \Pr[r_1 = z] = \frac{\max_{z \in \mathbb{F}_p} |\{k | H_{\mathbb{G}_{\text{in}},1}(\text{msg})^k \cdot X = z \wedge k \in D_k\}|}{|D_k|} \quad (3)$$

$$\leq \frac{\max_{z \in \mathbb{F}_p} |\{k | H_{\mathbb{G}_{\text{in}},1}(\text{msg})^k \cdot X = z \wedge k \in \mathbb{F}_p\}|}{|D_k|} \quad (4)$$

$$\leq \frac{\max_{z \in \mathbb{F}_p} |\{k | H_{\mathbb{G}_{\text{in}},1}(\text{msg})^k \cdot X = z \wedge k \in \mathbb{F}_{2hs}\}|}{|D_k|} \quad (5)$$

$$= \frac{4h}{|D_k|} = \frac{2}{s-1} \cdot 4h \quad (6)$$

$$\leq \frac{4h}{p-2h} \cdot 4h = \frac{16h^2}{p-2h} \quad (7)$$

Equation (3) follows from the definition of r_1 . (4) follows from the fact that $D_k \subset \mathbb{F}_p$. (5) is deduced from Hasse's theorem and holds for any $p \geq 12$. (6) follows from the fact that $\max_{z \in \mathbb{F}_p} |\{k | H_{\mathbb{G}_{\text{in}},1}(\text{msg})^k \cdot X = z \wedge k \in \mathbb{F}_{\tilde{h}s}\}| = 2\tilde{h}$ for any $\tilde{h}$ (which is true due to the fact that there are at most 2 group elements in $\mathbb{G}_{\text{in}}$ that share the same x -coordinate and the order of $\mathbb{G}_{\text{in}}$ is s). (7), similar to (5), is also deduced from Hasse's theorem and holds for any $p \geq 12$ provided that $p > 2h$.

Consequently, the guessing probability of (r_1, r_2) is bounded by $\left(\frac{16h^2}{p-2h}\right)^2$, and we denote this by γ . Then from the leftover hash lemma, the statistical distance between the distribution of (β, r) and the uniform distribution on $\mathbb{F}_p^2$ is given by

$$\Delta \leq \frac{\sqrt{p}}{2} \cdot \sqrt{\gamma} = \frac{8h^2\sqrt{p}}{p-2h} \approx \frac{1}{\sqrt{p}}$$

where h is the cofactor such that $|E(\mathbb{F}_p)| = h \cdot s$, and so this is our distinguishing advantage between Game 1 and Game 2:

$$|\Pr[W_2] - \Pr[W_1]| \leq \frac{8h^2\sqrt{p}}{p-2h} = \mathcal{O}\left(\frac{1}{\sqrt{p}}\right) \quad (8)$$

Finishing the proof. From (2) and (8), we conclude that the adversary's advantage is bounded as follows:

$$\text{Adv}_{\text{eVRF}^\dagger, \mathcal{A}}^{\text{psdr}}(\lambda) = |\Pr[W_0] - \Pr[W_2]| \leq \text{Adv}_{\text{DH}, \mathcal{B}_1}^{\text{unrec}}(\lambda) + \frac{8h^2\sqrt{p}}{p-2h}$$

We also note that extracting from $\tilde{n}$ samples (e.g. evaluating $\text{eVRF}^\dagger$ multiple times for $\tilde{n}$ different values of msg) increases Δ by a factor $\tilde{n}$, which is still at most negligible if $\tilde{n}$ is polynomial in λ . $\square$

E A Different Approach in Proof Construction

A different proof construction is possible if we forgo the proof of discrete logarithm (for the last exponentiation of $\mathcal{R}_{\text{eVRF}^\dagger}$) and instead represent $\mathcal{R}_{\text{eVRF}^\dagger}$ in its entirety as A, B, C matrices for Bulletproofs, albeit with a slight modification of the ninth step to be $R \leftarrow g_{\text{in}}^r \in \mathbb{G}_{\text{in}}$. The subtle advantage of this construction is that we can use the same group for the Diffie-Hellman key exchange and the eVRF commitment output, i.e. R is a pseudorandom group element in $\mathbb{G}_{\text{in}}$ not $\mathbb{G}_{\text{out}}$. While a second group $\mathbb{G}_{\text{out}}$ is required for the Bulletproofs argument system either way, this may be desirable in use cases where the PKI group and the DKG group must be equal for a more intuitive implementation.

The disadvantage is the increased number of constraints. Not only the modified ninth step (needing each iteration of the first and second gadgets amounting to $4\lambda + 4$ additional constraints), the eighth step would also add overhead. To ensure pseudorandomness, the leftover hash lemma must then be applied with a prime modulus different from the one used by all other constraints (such operation is called a *non-native arithmetic*), which can add a few thousands of constraints.

F Public Key Infrastructure

We give a proof of security in Section 6 for the protocol $\Pi_{\text{Golden-PKI}}$, which is Golden composed with an idealized PKI functionality.

We next introduce this ideal PKI functionality $\mathcal{F}_{\text{PKI}}$. We then give a concrete construction that realizes this functionality. Note that our ideal functionality assumes discrete-logarithm keys, which is required for $\Pi_{\text{Golden-PKI}}$.

F.1 Idealized Functionality $\mathcal{F}_{\text{PKI}}$

We show $\mathcal{F}_{\text{PKI}}$ in Figure 7. The functionality maintains two sets $Q_{\text{K}}^{(1)}$ and $Q_{\text{K}}^{(2)}$. The set $Q_{\text{K}}^{(1)}$ tracks the keys honestly generated by honest parties and corrupt parties that wish to follow the protocol. The functionality also maintains a set $Q_{\text{K}}^{(2)}$ of all registered public keys associated with participant identifiers. To register a public key PK, the adversary must query **Register** with the public key PK and associated identifier id. Additionally, the associated secret key sk must be provided as input. The oracle first authenticates the query, outputting $\perp$ if a corrupt party attempts to register a public key for any other party but themselves. Then, the oracle checks that (sk, PK) satisfies the relation $\mathcal{R}_{\text{PKI}}$, outputting $\perp$ if the relation does not hold. Otherwise, the functionality adds (PK, id) to the table Q_{K} .

To generate public and private keys, honest parties may query **KeyGen**. Corrupted parties that wish to follow the protocol honestly may also query this oracle.

Note that $\mathcal{F}_{\text{PKI}}$ is similar to the bulletin-board functionality $\bar{\mathcal{G}}_{\text{BB}}$ given by Canetti et al. [18]. In $\bar{\mathcal{G}}_{\text{BB}}$, the functionality accepts only the first registered value for an identity, and does not allow modification or deletion of the value. Also

similarly, $\bar{g}_{\text{BB}}$ is authenticated, and a user can only generate and register keys for themselves. However, our $\mathcal{F}_{\text{pki}}$ performs an additional validity check that the input values being registered satisfy the relation $\mathcal{R}_{\text{pki}}$.

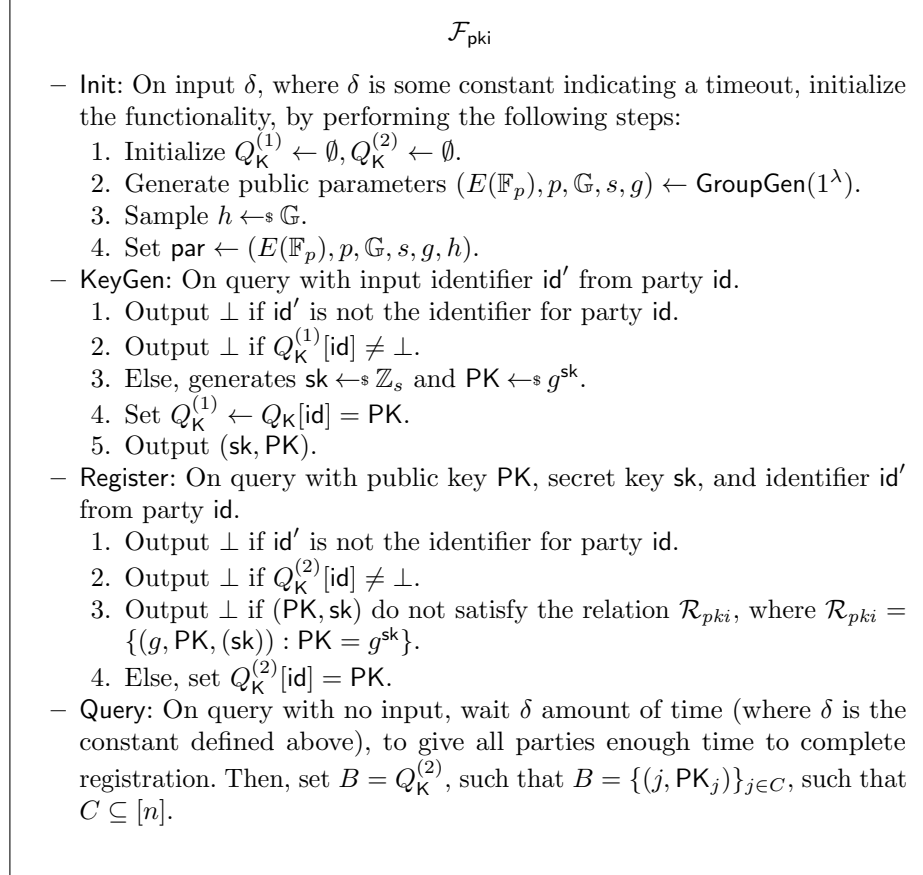


Fig. 7: Ideal functionality $\mathcal{F}_{\text{pki}}$ for a public-key infrastructure with proof of possession of the corresponding secret key sk . Public parameters par are given implicitly to all algorithms. The functionality authenticates participant queries, ensuring that parties can only generate and register keys for themselves.

F.2 A Concrete Protocol Realizing $\mathcal{F}_{\text{pki}}$

To realize $\mathcal{F}_{\text{pki}}$, we need a zero-knowledge proof system to demonstrate knowledge of sk . However, this zero-knowledge proof system must be online-extractable, to prove security in the UC framework, as rewinding is not allowed.

One possible direction is to use zero-knowledge proofs with Fischlin’s transform [41], which can be proven secure using standard assumptions. However, doing so introduces additional technical and performance complexity.

A Lightweight Protocol in the CRS Model. We next introduce a concrete protocol that realizes $\mathcal{F}_{\text{pki}}$ in a lightweight manner. We show this protocol in Figure 8.

The zero-knowledge proof relation $\mathcal{R}_P$ given in Figure 8 demonstrates that an ElGamal ciphertext is correct, and either that the prover knows the message that is encrypted (which is sk), or the prover knows the discrete logarithm of a public value h . Because h is chosen at random from $\mathbb{G}$ at the time of setup, then this proof by inference demonstrates that the prover knows sk , if the discrete logarithm problem is hard in $\mathbb{G}$.

It is straightforward to demonstrate that the concrete scheme in Figure 8 securely realizes $\mathcal{F}_{\text{pki}}$ in the UC framework, due to the non-interactive proof straight-line compiler given by Chen et al. [24]. Concretely, applying the compiler defined by Chen et al. [24] to a Schnorr proof of knowledge gives the Chaum-Pedersen zero-knowledge proof given in the concrete scheme in Figure 8.

G Simulation-Based Security for $\mathcal{R}_{\text{pki}}$

We next describe an idealized functionality for the relation $\mathcal{R}_{\text{pki}}$, which is given in Appendix F, in Figure 7 specifically. We define $\mathcal{F}_{\text{zk}}^{\mathcal{R}_{\text{EF}}}$ separately, as a game hop in our proof for Golden.

The ideal functionality $\mathcal{F}_{\text{zk}}^{\mathcal{R}_{\text{EF}}}$, following the definition by Boneh et al. [12], works as follows. The functionality receives a **Prove** message from party i with input $(i, \text{PK}, \text{sk})$. If $(\text{PK}, \text{sk}) \in \mathcal{R}_{\text{pki}}$, then the functionality broadcasts (i, PK) to all parties $j \in [n]$. Otherwise, the functionality returns $\perp$.

Note the **Prove** oracle in the definition by Boneh et al. [12] accepted as input a message $(i, j, \text{PK}, \text{sk})$, and if the input satisfied the relation, would send the message to party j . We instead give our definition where the **Prove** oracle broadcasts this message as opposed to delivering the message specifically to a single party.

H Proof for Theorem 3

Proof. We establish the proof by showing that $\text{Real}_{\Pi_{\text{Golden-PKI}}, \mathcal{A}}$ is indistinguishable from $\text{Ideal}_{\mathcal{F}_{\text{KeyGen}}^{\Delta}, \mathcal{S}}$, by a sequence of games.

Without loss of generality, we assume the adversary completes the Initialize step of the protocol correctly., and so the DKG protocol runs with the full set of n participants. Note the proof can easily be extended to the setting where only a subset of $\ell < n, \ell \geq t$ parties completed the Initialize step correctly, by simply performing the DKG among this subset of ℓ parties.

Game 0. This is the indistinguishability game $\text{Real}_{\Pi, \mathcal{A}}$ defined in Section 3.1 instantiated with $\Pi = \Pi_{\text{Golden-PKI}}$.

- **Setup**: Accepts as input public parameters par , and a timeout constant δ .
 1. Initialize $Q_K^{(1)} \leftarrow \emptyset, Q_K^{(2)} \leftarrow \emptyset$.
 2. Generate public parameters $(E(\mathbb{F}_p), p, \mathbb{G}, s, g) \leftarrow \text{GroupGen}(1^\lambda)$.
 3. Sample $h \leftarrow \mathbb{G}$.
 4. Set $\text{par} \leftarrow (E(\mathbb{F}_p), p, \mathbb{G}, s, g, h)$.
- **KeyGen**: If $Q_K^{(1)}[\text{id}] \neq \perp$, return $\perp$. Else, sample $\text{sk} \leftarrow \mathbb{Z}_s$, set $\text{PK} \leftarrow g^{\text{sk}}$. Set $Q_K^{(1)} \leftarrow Q_K[\text{id}] = \text{PK}$. Output (PK, sk) .
- **Register**: Accepts as input public key PK , identifier id' from party id , and proof π .
 1. Output $\perp$ if id' is not the identifier for party id .
 2. Output $\perp$ if $Q_K^{(2)}[\text{id}] \neq \perp$.
 3. Output $\perp$ if $\text{Verify}(\pi, \text{PK}) \neq 1$, where π proves the relation $\mathcal{R}_P$, as follows:

$$\mathcal{R}_P = \left\{ (g, h, \text{PK}, c = (C_0, C_1), z, R_g, R_h) ; (\text{sk}, r) \right\} \quad \text{iff}$$
 - (1) $C_0 = g^r ; C_1 = h^r \cdot g^{\text{sk}}$
 - (2) $g^z = R_g \cdot (g^{\text{sk}})^{c'}$
 - (3) $h^z = R_h \cdot (h^{\text{sk}})^{c'}$
 4. Else, set $Q_K^{(2)}[\text{id}] = \text{PK}$.
- **Query**: Wait δ amount of time (where δ is the constant defined above), to give all parties enough time to complete registration. Then, output $Q_K^{(2)}$.

Fig. 8: Concrete protocol that realizes $\mathcal{F}_{\text{pki}}$, as defined in Figure 7. Public parameters par are given implicitly to all algorithms. Specifically, the concrete protocol uses a zero-knowledge proof π to demonstrate that the owner of a public key PK knows the corresponding secret key. Verify checks that the π satisfies the relation $\mathcal{R}_P$. This relation allows for straight-line extractability in the CRS model, a requirement to demonstrate UC security. We assume requests can be authenticated to a user's identity id using some external mechanism.

Game 1. In Game 1, we make the following modifications. Let $\mathcal{F}_{\text{zk}}^{\mathcal{R}_{EF}}$ be the idealized functionality for a zero-knowledge proof, as further described in Appendix G. $\mathcal{F}_{\text{zk}}^{\mathcal{R}_{EF}}$ enforces the relation $\mathcal{R}_{EF} = \{(\text{PK}, (\text{sk}, r)) : \text{KeyGen}(1^\lambda; r) = (\text{sk}, \text{PK})\}$.

In Game 1, all parties instead perform a different initialize step as follows. We call the union of **Golden** with this different initialize step $\Pi_{\text{Golden-eVRF}}$. Overall, instead of each party querying the $\mathcal{F}_{\text{pki}}$ oracles **KeyGen**, **Register**, **Query**, each party i instead does the following:

Protocol 2 ($\Pi_{\text{Golden-eVRF}}$):

- **Initialize:** Each party $i \in [n]$ interacts with the eVRF functionality $\mathcal{F}_{\text{eVRF}}$ as defined in Appendix A with the following steps:
 1. Sample $\text{sk}_i^I \leftarrow \mathbb{Z}_s$, derive $\text{PK}_i^I \leftarrow g^{\text{sk}_i^I}$.
 2. Query **Register**(PK_i^I, i) to $\mathcal{F}_{\text{eVRF}}$.
 3. Query **Query** to $\mathcal{F}_{\text{eVRF}}$, receive $B = \{(j, \text{PK}_j^I)\}_{j \in C}$, such that $C \subseteq [n]$.
 4. Query **Prove**($i, \text{PK}_i^I, \text{sk}_i^I$) to $\mathcal{F}_{\text{zk}}^{\mathcal{R}_{EF}}$, receive $\hat{B} = \{(j, \hat{\text{PK}}_j^I)\}_{j \in C'}$, such that $C' \subseteq [n]$, from the functionality.
 5. For each $(i, \text{PK}_i^I) \in B, (i, \hat{\text{PK}}_i^I) \in \hat{B}$: if $\text{PK}_i^I = \hat{\text{PK}}_i^I$, store (i, PK_i^I) .

Difference between Game 0 and Game 1. The difference between Game 0 and Game 1 is bounded by the indistinguishability of the initialize step for $\Pi_{\text{Golden-PKI}}$ and the initialize step described above for $\Pi_{\text{Golden-eVRF}}$.

It is easy to see why this difference is zero. First, recall that Boneh et al. [12] give a proof of equivalence (Theorem 2) between the idealized functionality $\mathcal{F}_{\text{eVRF}}$ and the game-based notions of pseudorandomness and simulatability, for a protocol Π_{EF} , which composes an eVRF with an initialize step that all parties perform at the start of the protocol. This initialize step is equivalent to what is described for $\Pi_{\text{Golden-eVRF}}$.

In the discrete logarithm setting, $\mathcal{R}_{EF}$ is identical to the relation $\mathcal{R}_{\text{pki}}$ that our $\mathcal{F}_{\text{pki}}$ enforces for **KeyGen**, (as both require proving knowledge of the secret key corresponding to the participant's public key). Thus, querying the **Prove** oracle for $\mathcal{F}_{\text{zk}}^{\mathcal{R}_{EF}}$ gives no additional advantage, instead of this relation being checked within the **Register** oracle for $\mathcal{F}_{\text{pki}}$. Further, because the **Register** and **Query** oracles for $\mathcal{F}_{\text{pki}}$ and $\mathcal{F}_{\text{eVRF}}$ are equivalent, then employing the $\mathcal{F}_{\text{eVRF}}$ oracles gives no additional advantage to the adversary.

Hence,

$$\Pr[W_1] = \Pr[W_0]$$

Game 2. In Game 2, we make the following modifications.

In Round 0 of the DKG, each party i queries $\mathcal{F}_{\text{eVRF}}$ on its **Evaluate** oracle (as defined in Figure 5) n times on inputs $(i, \text{msg}_{i,j} = "(\text{PK}_j^I, \text{msg}_i)")$ for $j \in [n]$, and receives $(r_{i,j}, R_{i,j}), j \in [n]$ in return. Each party then follows the remainder of Round 0 as in Game 1. In Round 1, each party similarly queries $\mathcal{F}_{\text{eVRF}}$ n times on inputs $(j, \text{msg}_j = "(\text{PK}_i^I, \text{msg}_j)")$ for $j \in [n]$, receiving $(r_{j,i}, R_{j,i}), j \in [n]$ in return. Recall that in $\mathcal{F}_{\text{eVRF}}$, inputs and outputs are generated and delivered to all parties honestly, and so parties do not need additional zero-knowledge proofs of correctness or to perform **Verify** on the outputs.

Difference between Game 1 and Game 2. The difference between Game 1 and Game 2 is bounded by the indistinguishability of $\text{eVRF}^\dagger$ from $\mathcal{F}_{\text{eVRF}}$. We show $\text{eVRF}^\dagger$ is a secure eVRF, as shown in the proof for Theorem 2.

Because Game 1 employs an equivalent initialize functionality as π_{EF} , then $\text{eVRF}^\dagger$ in conjunction with this initialize functionality satisfies the requirements

for this equivalence to hold, as shown by the proof of equivalence (Theorem 2) by Boneh et al. [12].

Hence,

$$\Pr[W_2] = \Pr[W_1]$$

Game 3. We next describe a simulating algorithm **SIM** that interacts with $\mathcal{A}$ in a black-box manner. **SIM** accepts as input first parameters (n, t) , and then a random challenge $Y \leftarrow \mathbb{G}$, which **SIM** does not know the discrete logarithm to. **SIM** simulates $\mathcal{F}_{\text{eVRF}}$ and $\mathcal{F}_{\text{zk}}^{\mathcal{R}_{EF}}$. We describe **SIM** in detail next.

Initialize. On input (n, t) , **SIM** does the following.

1. Send (n, t) to $\mathcal{A}$, receive **corr** in return. Output **corr**.
2. Receive as input $Y \in \mathbb{G}$.
3. Set $\text{hon} \leftarrow [n] \setminus \text{corr}$.
4. Initialize the tables $Q_K^{(1)} \leftarrow \emptyset$, $Q_K^{(2)} \leftarrow \emptyset$, $Q_{K'} \leftarrow \emptyset$, $Q_{\text{evrf}} \leftarrow \emptyset$.

Participant Initialization. **SIM** simulates the initialize step for each participant as defined in Game 2, as follows.

1. For all honest parties $i \in \text{hon}$, **SIM** performs key generation honestly, sampling $\text{sk}_i^I \leftarrow \mathbb{Z}_s$, and setting $\text{PK}_i^I \leftarrow g^{\text{sk}_i^I}$.
2. **SIM** then simulates each honest party $i \in \text{hon}$ querying **Register**, by setting $Q_K^{(1)}[\text{PK}_i^I] \leftarrow i$.
3. When the adversary queries **Register** on input (j, f_j) where $j \in \text{corr}$, **SIM** follows the actions of $\mathcal{F}_{\text{eVRF}}$ exactly, by setting $Q_K^{(2)}[\text{PK}_j^I] = (j, f_j)$ if $Q_{\text{reg}}[j]$ has not already been set, and similarly for $Q_K^{(1)}$.
4. When the adversary queries **Query** for $\mathcal{F}_{\text{eVRF}}$, **SIM** follows the steps as in the ideal functionality exactly, outputting the contents of $Q_K^{(1)}$.
5. When the adversary queries **Prove** on input $(j, \text{PK}_j^I, \text{sk}_j^I)$, **SIM** updates $Q_{K'} \leftarrow Q_{K'} \cup \{(\text{PK}_j^I, j, \text{sk}_j^I)\}$, and otherwise follows the actions of $\mathcal{F}_{\text{zk}}^{\mathcal{R}_{EF}}$ exactly.

Simulating $\mathcal{F}_{\text{eVRF}}$ Evaluate. **SIM** simulates the $\mathcal{F}_{\text{eVRF}}$ Evaluate oracle as defined in Figure 5 as follows:

1. For corrupt parties $k \in \text{hon}$, when $\mathcal{A}$ queries **Evaluate** on input (k, msg) , **SIM** follows the actions of $\mathcal{F}_{\text{eVRF}}$ exactly, setting $A \leftarrow g^{f_k(\text{msg})}$. Finally, **SIM** outputs A .
2. When **SIM** (later in the simulation of Round 1) simulates honest parties' interactions with $\mathcal{F}_{\text{eVRF}}$ for honest parties $j \in \text{hon}, j \neq \tau$, **SIM** follows the actions of $\mathcal{F}_{\text{eVRF}}$ exactly. **SIM** picks $\alpha \leftarrow \mathbb{Z}_p$, and sets $A \leftarrow g^\alpha$. **SIM** then updates $Q_{\text{evrf}}[(\text{id}, \text{msg})] \leftarrow \alpha$, and outputs (α, A) .

Simulating Key Generation. Next, **SIM** must simulate the protocol on behalf of honest parties to $\mathcal{A}$. We show how **SIM** does that next.

Round 0. To simulate honest parties' contributions for round 0 of **Golden**, **SIM** does the following:

1. For each honest party $k \in \text{hon}, k \neq \tau$, **SIM** follows the protocol in the same manner as defined in Game 2, and simulates $\mathcal{F}_{\text{eVRF}}$ for honest parties as defined above.
2. However, for the last honest party queried $\tau \in \text{hon}$, **SIM** simulates the protocol as follows:
 - (a) Follows the protocol honestly to sample a message $\text{msg}_\tau \leftarrow \{0, 1\}^\lambda$.
 - (b) Randomly sample corrupt parties' shares $\bar{x}_{\tau,j} \leftarrow \mathbb{Z}_p$ for all $j \in \text{corr}$.
 - (c) **SIM** then simulates the polynomial commitment $\bar{C}_\tau$ to the polynomial f_τ , as follows:
 - i. Set

$$A_{\tau,0} \leftarrow Y \cdot \prod_{k \in \text{hon}, k \neq \tau} (A_{k,0})^{-1} \quad (9)$$

where $A_{k,0}$ are from the honest parties $k \in \text{hon}, k \neq \tau$ polynomial commitments $\bar{C}_k$.

- ii. **SIM** computes the Lagrange polynomials $\{L'_0(Z), \{L'_j(Z)\}_{j \in \text{corr}}\}$ for the set (of x -coordinates) $\{0 \cup \text{corr}\}$, as defined in Section 3.3.
- iii. For $i \in \{1, \dots, t-1\}$, **SIM** then sets the remaining commitments of f_τ "in the exponent", as follows:

$$A_{\tau,i} \leftarrow A_{\tau,0}^{L'_0(i)} \cdot \prod_{j \in \text{corr} \cup \{0\}} (g^{\bar{x}_{\tau,j}})^{L'_j(i)}$$

- iv. Set $\bar{C}_\tau = (A_{\tau,0}, \dots, A_{\tau,t-1})$. Here, $\bar{C}_\tau$ is the commitment to f_τ , defined by the $t-1$ corrupt parties' shares, and Y , such that $g^{f_\tau(0)} = A_{\tau,0}$ as given in Equation 9.
- (d) For each corrupt party $j \in \text{corr}$, simulate $\mathcal{F}_{\text{eVRF}}$ on inputs $(\tau, \text{msg}_{\tau,j} = \text{"(PK}_j^I, \text{msg}_\tau)\text{"})$ for $j \in \text{corr}$, as defined above. Then, set $z_{\tau,j} \leftarrow r_{\tau,j} + \bar{x}_{\tau,j}$.
- (e) For $\ell \in \text{hon}$, define

$$\bar{X}_{\tau,\ell} \leftarrow A_{\tau,0} \cdot \prod_{i=1}^{t-1} A_{\tau,i}^{\ell^i} \quad (10)$$

to be the commitment to the (unknown) ℓ^{th} honest party's share $\bar{x}_{\tau,\ell}$.

- (f) Then, simulate the encryption $z_{\tau,\ell}$ to honest parties' $\ell \in \text{hon}$ shares (which are unknown) by performing the following steps:
 - i. Sample $z_{\tau,\ell} \leftarrow \mathbb{Z}_p$.
 - ii. Set

$$R_{\tau,\ell} \leftarrow g^{z_{\tau,\ell}} \cdot \bar{X}_{\tau,\ell} \quad (11)$$

- (g) For all parties $j \in [n]$, follow the protocol to set $\sigma_{\tau,j} \leftarrow (R_{\tau,j}, z_{\tau,j})$.
 - (h) Set $\text{bmsg}_\tau \leftarrow (\text{msg}_\tau, \bar{C}_\tau, \{\sigma_{\tau,j}\}_{j \in [n]})$.
3. Output $\{\text{bmsg}_j\}_{j \in \text{hon}}$ for all honest parties.

Round 1. When $\mathcal{A}$ queries honest parties on Round 1 with inputs $\{\text{bmsg}_j\}_{j \in \text{corr}}$, **SIM** does not output anything (as is the case in the real protocol).

Analysis. The analysis of **SIM** as compared to Game 2 is based on the following:

- (1) Simulation of $\mathcal{F}_{zk}^{\mathcal{R}_{EF}}$
- (2) Simulation of $\mathcal{F}_{eVRF}$
- (3) Simulation of honest parties' outputs of Round 0
- (4) Correctness of corrupt parties' checks in Round 1

For (1), **SIM**'s simulation of $\mathcal{F}_{zk}^{\mathcal{R}_{EF}}$ is indistinguishable from Game 2, because **SIM** follows the same steps as the idealized $\mathcal{F}_{zk}^{\mathcal{R}_{EF}}$, but simply maintains state in Q_K .

For (2), **SIM**'s simulation of $\mathcal{F}_{eVRF}$ is indistinguishable from the ideal functionality as defined in Figure 7, for the following reasons. First, **SIM** follows the same **Register** steps as the ideal functionality. Secondly, for **Evaluate** queries for all parties $k \in \text{hon}, k \neq \tau$ and for corrupt parties, **SIM** simulates $\mathcal{F}_{eVRF}$ in the same manner as the ideal functionality. For the simulated honest party $\tau \in \text{hon}$, **SIM**'s simulation of $\mathcal{F}_{eVRF}$ is likewise perfect, because $R_{\tau,\ell} = g^{z_{\tau,\ell}} \cdot \bar{X}_{\tau,\ell}$ as defined as in Equation 11, where $\bar{X} = g^{f_\tau}(\ell)$, as defined in Equation 10. Because $z_{\tau,\ell}$ is sampled uniformly at random from $\mathbb{Z}_p$, then $g^{z_{\tau,\ell}}$ will likewise be a uniformly random element in $\mathbb{G}$, and hence so will be $g^{z_{\tau,\ell}} \cdot \bar{X}_{\tau,\ell}$. Consequently, $R_{\tau,\ell}$ for all $\ell \in \text{hon}$ will be indistinguishable from $\mathcal{F}_{eVRF}$'s real output for an honest party.

For (3), **SIM**'s simulation of honest parties' outputs from Round 0 is indistinguishable from Game 2 for the following reasons:

- When generating Shamir secret shares for τ , corrupt parties receive random shares $\bar{x}_{\tau,j} \leftarrow \mathbb{Z}_p$, which are $t - 1$ points on a polynomial f_τ , such that $g^{f_\tau} = \bar{C}_\tau$, as shown in Equation 9.
- For all honest parties $j \in \text{hon}, j \neq \tau$, **SIM** simulates $\mathcal{F}_{eVRF}$ exactly as in Game 2.
- However, for the honest party $\tau \in \text{hon}$, **SIM**'s simulation is indistinguishable to $\mathcal{F}_{eVRF}$,

Lastly, for (4), corrupt parties that follow the protocol will receive valid outputs in Round 1, for the following reasons:

- Due to how $R_{\tau,\ell}$ for $\ell \in \text{hon}$ is defined above, the check $g^{z_{\tau,\ell}} = R_{\tau,\ell} \cdot \bar{X}_{\tau,\ell}$ will hold by definition. All other honest parties' values $(R_{i,j}, z_{i,j})$ for $i \in \text{hon}, j \in [n]$ are defined by following the protocol.
- Each $\text{PK}_i = g^{\bar{x}_i}, i \in [n]$ will be valid and indistinguishable from a real run of the protocol, because $\text{PK}_i = \prod_{j \in [n]} \bar{X}_{j,i}$. Although f_τ is unknown, $\bar{C}_\tau$ commits to f_τ “in the exponent”, and f_τ is fully defined by the discrete logarithm of Y , and all other honest parties' contributions. Although **SIM** does not know $\bar{x}_{\tau,i}$ directly, it can derive $\bar{X}_{\text{simid},i}$, simply by performing polynomial interpolation “in the exponent.”
- For similar reasons as above, PK will likewise be valid and indistinguishable from a real run of the protocol.

Output. **SIM** outputs the public key PK , and the corrupt parties' contributions to the DKG $\{\omega_j\}_{j \in \text{corr}}$.

Finishing the Proof. At the end of SIM's simulation, to show indistinguishability from $\mathcal{F}_{\text{KeyGen}}^\Delta$, we must show that the public key PK is an additive shift of Y from $\mathcal{F}_{\text{KeyGen}}^\Delta$ with respect to Δ , where Δ is an additive shift defined by the adversary. Furthermore, we must show that $\text{PK}_i = g^{\text{sk}_i}$, $i \in [n]$, where $\text{sk}_i = f(i)$, such that $g^{f(0)} = \text{PK}$, and $f(j) = \text{sk}_j$, for $j \in \text{corr}$.

SIM can derive $\Delta = \sum_{j \in \text{corr}} \omega_j$ (the adversary's contributions to the DKG secret key) by the following steps:

1. Obtain $(\text{sk}_k^I)_{k \in \text{corr}}$ from $Q_{K'}$.
2. Derive $(r_{j,i}, \cdot, \cdot) \leftarrow \text{eVRF}^\dagger.\text{Evaluate}(\text{sk}_j^I, (\text{msg}_j, \text{PK}_i^I))$ for each $j \in \text{corr}, i \in [n]$, using its knowledge of $\text{sk}_j^I, j \in \text{corr}$.
3. SIM then derives $\bar{x}_{j,i} \leftarrow z_{j,i} - r_{j,i}$, for each $j \in \text{corr}, i \in [n]$.
4. SIM then derives $\omega_j \leftarrow \text{Shamir.Recover}(t, \{i, \bar{x}_{j,i}\}_{i \in [n]})$, for each $j \in \text{corr}$.
5. Finally, set $\Delta \leftarrow \sum_{j \in \text{corr}} \omega_j$.

The public key derived by all parties at the end of Round 1 is as follows:

$$\text{PK} \leftarrow \prod_{j=1}^n \text{PK}_j$$

Expanding this out, we get:

$$\begin{aligned}
\text{PK} &= \prod_{j=1}^n \text{PK}_j \\
\text{PK} &= \prod_{j=1}^n A_{j,0} \\
\text{PK} &= A_{\tau,0} \cdot \prod_{j \in [n], j \neq \tau} A_{j,0} \\
\text{PK} &= Y \cdot \prod_{k \in \text{hon}, k \neq \tau} (A_{k,0})^{-1} \cdot \prod_{j \in [n], j \neq \tau} A_{j,0} \quad // \text{ From Equation 9} \\
\text{PK} &= Y \cdot \prod_{k \in \text{corr}} A_{k,0} \quad // \text{ Cancelling out honest parties' contributions} \\
\text{PK} &= Y \cdot g^\Delta
\end{aligned} \tag{12}$$

As shown in the relation in Equation 12 between PK, the challenge Y , and corrupt parties' contributions $\{\omega_j\}_{j \in \text{corr}}$, SIM's output in Game 3 is indistinguishable from $\text{Ideal}_{\mathcal{F}_{\text{KeyGen}}^\Delta, \mathcal{S}}$. This completes the proof. $\square$